

APPLICAZIONE MATCHUP

CORSO DI INGEGNERIA DEL SOFTWARE A.A. 2025 / 2026

DOCUMENTO DI PROGETTO FINALE

SALVATORE CAMPIONE

MARIO CIPRIANO

Sommario

1	IDEAZIONE E ANALISI DEI REQUISITI	4
1.1	INTRODUZIONE	4
1.2	REQUISITI	4
1.3	OBIETTIVI E CASI D'USO	5
1.4	MODELLO DEI CASI D'US	5
UC1:	REGISTRAZIONE STRUTTURA	5
UC2:	REGISTRAZIONE GIOCATORE	7
UC3:	EFFETTUA PRENOTAZIONE	8
UC4:	CREAZIONE NUOVO TORNEO	10
UC5:	CREAZIONE SQUADRA	11
UC6:	ISCRIZIONE TORNEO	13
UC7:	GENERA CALENDARIO TORNEO	15
UC8:	ASSEGNAZIONE ESITO PARTITA	16
UC9:	ANNULLAMENTO PRENOTAZIONE	18
UC10:	EFFETTUA RICARICA CONTO	19
UC11:	EFFETTUA RICARICA CONTO GIOCATORE	20
1.5	DOCUMENTO DI VISIONE	22
1.6	REGOLE DI BUSINESS	23
1.7	SPECIFICHE SUPPLEMENTARI	26
1.8	GLOSSARIO	26
2.	ANALISI	27
2.1	INTRODUZIONE	27
2.2.	MODELLO DI DOMINIO FINALE	29
2.3	DIAGRAMMI DI SEQUENZA DI SISTEMA E CONTRATTI DELLE OPERAZIONI	30
2.3.1	DIAGRAMMA DI SEQUENZA DI SISTEMA UC1	30
2.3.2	DIAGRAMMA DI SEQUENZA DI SISTEMA UC2	33
2.3.3	DIAGRAMMA DI SEQUENZA DI SISTEMA UC3	34
2.3.4	DIAGRAMMA DI SEQUENZA DI SISTEMA UC4	37
2.3.5	DIAGRAMMA DI SEQUENZA DI SISTEMA UC5	38
2.3.6	DIAGRAMMA DI SEQUENZA DI SISTEMA UC6	40
2.3.7	DIAGRAMMA DI SEQUENZA DI SISTEMA UC7	41
2.3.8	DIAGRAMMA DI SEQUENZA DI SISTEMA UC8	42

2.3.9 DIAGRAMMA DI SEQUENZA DI SISTEMA UC9	44
2.3.10 DIAGRAMMA DI SEQUENZA DI SISTEMA UC10.....	45
2.3.11 DIAGRAMMA DI SEQUENZA DI SISTEMA UC11.....	47
3 PROGETTAZIONE	48
3.1 INTRODUZIONE	48
3.2 DIAGRAMMI DI SEQUENZA	49
3.2.1 PRENOTA CAMPO.....	49
3.2.2 CALCOLA COSTO TOTALE	49
3.2.3 CONFERMA PRENOTAZIONE	50
3.2.4 INSERISCI DATI TORNEO	50
3.2.5 CREA SQUADRA.....	50
3.2.6 CERCA TORNEO	51
3.2.7 EFFETTUA ISCRIZIONE TORNEO.....	51
3.2.8 GENERA CALENDARIO.....	52
3.2.9 DEFINISCI GIORNATE CALENDARIO	52
3.2.10 ASSEGNA ESITO PARTITA E AGGIORNAMENTO CLASSIFICA	53
3.2.11 ANNULLA PRENOTAZIONE	53
3.2.12 CALCOLA RIMBORSO	54
3.2.13 EFFETTUA RIMBORSO	54
3.3 DIAGRAMMA DELLE CLASSI DI PROGETTO	55
4 PATTERN UTILIZZATI NELLO SVILUPPO DEL PROGETTO.....	55
5 FASE DI TESTING	57
5.1 INTRODUZIONE AL TESTING	57
5.2 SCENARIO REGISTRAZIONE GIOCATORE	58
5.3 SCENARIO REGISTRAZIONE STRUTTURA	59
5.4 SCENARIO PRENOTA CAMPO	60
5.5 SCENARIO DI CREAZIONE TORNEO	62
5.6 SCENARIO DI CREAZIONE DELLA SQUADRA	63
5.7 SCENARIO DI ISCRIZIONE AL TORNEO.....	63
5.8 SCENARIO DI ANNULLAMENTO PRENOTAZIONE	64
5.9 SCENARIO RICARICA CONTO	65
5.10 SCENARIO DI RICARICA CONTO GIOCATORE.....	65
5.11 MODIFICA SCENARIO DI TEST ISCRIZIONE TORNEO	66
5.12 SCENARIO DI GENERAZIONE DEL CALENDARIO DEL TORNEO.....	66
5.13 SCENARIO DI ASSEGNAZIONE ESITO ALLA PARTITA	67

1 IDEAZIONE E ANALISI DEI REQUISITI

1.1 INTRODUZIONE

Nella sezione ideazione viene mostrata una panoramica generale dell'idea di implementazione del software *MatchUp*, mostrando le varie caratteristiche dell'applicativo insieme ai requisiti di quest'ultimo. Gli elaborati presenti in questa sezione di documento sono:

- **Modello dei casi d'uso** per evidenziare le funzionalità dell'applicazione per ciascun utilizzatore;
- **Documento di visione** per avere una panoramica generale delle funzionalità e dei requisiti complessivi dell'applicativo;
- **Regole di business** necessarie per una corretta erogazione dei servizi da parte dell'applicativo;
- **Glossario** per avere una panoramica sui concetti base che l'applicativo adopera;

1.2 REQUISITI

MatchUp è un sistema software per la gestione di prenotazioni dei campi di calcetto che si trovano all'interno di strutture registrate sulla piattaforma. Il sistema prevede anche la possibilità per i gestori delle strutture di creare tornei a cui gli utenti potranno iscriversi e partecipare.

Le funzionalità sono espresse di seguito:

- Registrazione della struttura / utente presso la piattaforma;
- Prenotazione di specifici campi da calcio per una determinata fascia oraria;
- Calcolo del prezzo delle prenotazioni sulla base della fascia oraria scelta, sulla data e sulle prenotazioni effettuate precedentemente.
- Creazione di tornei pubblici di calcio nella struttura;
- Gestione dei tornei creati;
- Iscrizione di un utente/squadra presso un torneo;
- Annullamento di prenotazioni esistenti.

1.3 OBIETTIVI E CASI D'USO

Di seguito si riportano i casi d'uso individuati, specificando l'attore primario di ciascuno ed il relativo obiettivo da raggiungere

Attore	Obiettivo	Caso d'uso
Gestore	Registrare la propria struttura nel sistema	UC1: Registrazione struttura
Giocatore	Registrazione account nel sistema	UC2: Registrazione giocatore
Giocatore	Selezionare e bloccare uno campo della struttura in una fascia oraria disponibile.	UC3: Effettua prenotazione
Gestore	Configurare e pubblicare un nuovo torneo di calcetto che avviene nel centro sportivo.	UC4: Creazione nuovo torneo
Giocatore	Creazione e registrazione della squadra per la partecipazione al torneo.	UC5: Creazione Squadra
Gestore	Richiedere la partecipazione a un torneo pubblico esistente.	UC6: Iscrizione torneo
Gestore	Organizzare la sequenza di incontri del torneo.	UC7: Genera calendario torneo
Gestore	Assegnare l'esito della partita e i punti alle squadre	UC8: Assegnazione esito partita
Giocatore	Revocare una prenotazione precedentemente effettuata.	UC9: Annullamento prenotazione
Giocatore	Ricaricare il proprio conto personale	UC10: Effettua ricarica conto
Gestore	Ricaricare conto dei giocatori che utilizzano il servizio	UC11: Effettua ricarica conto giocatore

1.4 MODELLO DEI CASI D'US

Viene qui presentata una descrizione dei casi d'uso; in particolare, partendo dalla fase di ideazione in cui la maggior parte dei casi d'uso erano descritti in formato breve, seguendo il processo iterativo si è passati gradualmente ad una descrizione via via più dettagliata.

UC1: REGISTRAZIONE STRUTTURA

Il gestore della struttura, dopo aver inserito i suoi dati personali per registrarsi nel sistema, vuole registrare il proprio centro sportivo all'interno del sistema MatchUp.

Campo	Descrizione
Nome del caso d'uso	Registrazione struttura
Portata	Applicazione MatchUp
Livello	Obiettivo utente
Attore primario	Gestore della struttura
Parti interessate	Gestore della struttura: desidera registrarsi nel sistema e registrare la propria struttura nella piattaforma per usufruire dei servizi di quest'ultima.
Pre-condizioni	1. Il gestore della struttura ha lanciato l'applicativo Matchup. 2. Il gestore della struttura non è registrato sulla piattaforma
Garanzia di successo	La struttura viene registrata su MatchUp
Scenario principale di successo	1. Il gestore della struttura seleziona la voce "Registra Struttura" 2. Il gestore inserisce le proprie generalità quali Nome, Cognome e Data di Nascita. 3. Il gestore sceglie le sue credenziali (username e password) 4. Il sistema chiede la conferma dei dati inseriti 5. Il gestore inserisce le informazioni sulla sua struttura quali nome della struttura, il luogo, giorni e orari di apertura e chiusura. 6. Il gestore inserisce i dati inerenti a un campo della struttura quali Tipologia , categoria di campo e tariffa oraria. il passo 6 viene ripetuto finché serve 7. Il Sistema chiede la conferma dei dati e registra la struttura con i campi
Estensioni	2a. La data di nascita inserita corrisponde ad un'età minore di 18 anni - Il sistema ritorna un messaggio di errore e torna indietro al menù principale 3a. Lo username inserito risulta già presente nel sistema - Il sistema segnala l'errore e viene richiesto di scegliere un nuovo username 3b. La password inserita non è valida - Il sistema segnala l'errore e viene richiesto di digitare una password valida 4a. Il gestore non conferma l'inserimento dei dati - Il sistema annulla l'inserimento e ritorna all'inizio dell'operazione di registrazione e l'utente deve reinserire i dati. 7a. Il gestore non conferma l'inserimento dei dati della struttura - Il sistema annulla l'inserimento in corso e ritorna all'inserimento dei dati della struttura.
Requisiti speciali	Inserimento di regole di business nei passi 2 e 3

Elenco delle varianti tecnologiche e dei dati	· Dati gestiti: • Dati personali del gestore e delle strutture e dei campi. · Persistenza: • I dati devono essere gestiti in memoria tramite strutture dati Java • La persistenza è garantita tramite file di testo (es. .txt o .csv)
Frequenza di ripetizione	Diverse volte al giorno
Varie	

UC2: REGISTRAZIONE GIOCATORE

Un giocatore vuole registrarsi al sistema per poter usufruire dei servizi di quest'ultimo.

Campo	Descrizione
Nome del caso d'uso	Registrazione giocatore
Portata	Applicazione MatchUp
Livello	Obiettivo utente
Attore primario	Giocatore
Parti interessate	Giocatore: Vuole registrarsi al sistema per usufruire dei servizi di quest'ultimo
Pre-condizioni	Il Giocatore non deve essere già registrato sulla piattaforma. Il Giocatore deve aver avviato l'applicativo.
Garanzia di successo	Il Giocatore viene registrato sul sistema.
Scenario principale di successo	1. Il Giocatore seleziona la voce "Registrazione utente" 2. Il Giocatore inserisce le proprie generalità quali Nome, Cognome e Data di Nascita. 3. Il Giocatore sceglie le sue credenziali (username e password) 4. Il Sistema chiede la conferma e registra il Giocatore sulla piattaforma

Estensioni	2a. La data di nascita inserita corrisponde ad un'età minore di 14 anni - Il Sistema ritorna un messaggio di errore e mostra il menù iniziale 3a. Il Giocatore inserisce uno username già esistente - Il Sistema segnala il problema e invita il Giocatore a reinserire un altro username 3b. Il Giocatore inserisce una password non valida - Il Sistema segnala l'errore e invita il Giocatore a inserire una password valida. 4a. Il giocatore non conferma l'inserimento dei dati
	- Il sistema annulla l'inserimento in corso e ritorna al passo 2
Requisiti speciali	Inserimento di regole di business nei passi 2 e 3
Elenco delle varianti tecnologiche e dei dati	· Dati gestiti: • Dati personali del giocatore · Persistenza: • I dati devono essere gestiti in memoria tramite strutture dati Java • La persistenza è garantita tramite file di testo (es. .txt o .csv).
Frequenza di ripetizione	Diverse volte al giorno
Varie	

UC3: EFFETTUA PRENOTAZIONE

Nome del caso d'uso	Effettua prenotazione
Portata	Applicazione MatchUp
Livello	Obbiettivo Utente
Attore Primario	Giocatore
Parti interessate	- Giocatore: Vuole effettuare una prenotazione di un campo da lui richiesto in una determinata struttura; - Gestore della struttura: vuole poter visualizzare le prenotazioni effettuate per i campi della sua struttura

Pre-condizioni	Il giocatore e la struttura devono essere registrati sulla piattaforma
Garanzia di successo	La prenotazione del giocatore viene confermata. Il campo scelto viene segnato come occupato per quel giorno e fascia oraria selezionata.

Scenario principale di successo	1. Il giocatore avvia l'applicazione ed effettua il login con le sue credenziali e seleziona effettua una prenotazione 2. Il giocatore cerca la struttura dove prenotare il campo, inserendo la categoria del campo. 3. Il giocatore seleziona il giorno e la fascia oraria desiderata in cui prenotare il campo . 4. Il sistema mostra al giocatore il riepilogo della prenotazione con il costo e chiede al giocatore di pagare. 5. Il giocatore conferma il pagamento e la prenotazione viene effettuata e inserita nel sistema
Estensioni	1a. Il giocatore digita le credenziali errate - Il giocatore prova a reinserire le sue credenziali 4a. Il giocatore paga in contanti - Il sistema emette la ricevuta di pagamento - Il giocatore si presenta con la ricevuta in struttura e va a pagare. 4b. Il giocatore paga con carta di credito - Il sistema chiede all'utente di inserire i dati della carta di credito. - Il giocatore inserisce i dati, verificandone la correttezza. a. Il pagamento è andato a buon fine.

	b. Il pagamento è stato rifiutato e si ripete la procedura dall'inizio.
Requisiti Speciali	Vengono inserite le regole di business dal passo 3 al passo 5
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizione	Diverse volte al giorno
Varie	

UC4: CREAZIONE NUOVO TORNEO

Nome del caso d'uso	Creazione Nuovo Torneo
Portata	Applicazione MatchUp
Livello	Obbiettivo Utente
Attore Primario	Gestore della Struttura
Parti interessate	- Gestore: Vuole creare e rendere pubblico un torneo nella propria struttura.
Pre-condizioni	- Il gestore della struttura avvia MatchUp - Il gestore della struttura si è autenticato
Garanzia di successo	Il torneo viene pubblicato e reso disponibile alla partecipazione dei giocatori.
Scenario principale di successo	1. Il Gestore seleziona la voce "Crea torneo dal menù" 2. Il gestore inserisce tutti di dati di creazione del torneo quali il nome, il tipo di torneo(calcio a 5,calcio a 7,ecc), il numero di componenti minimo per squadra, il costo di iscrizione e il giorno di scadenza delle iscrizioni e il giorno di inizio 3. Il sistema valida la correttezza formale dei dati e mostra al gestore il riepilogo 4. Il gestore conferma la creazione del torneo 5. Il sistema registra il nuovo torneo e lo rende visibile ai giocatori che si vogliono iscrivere
Estensioni	2a. Il gestore inserisce dei dati errati o relativi ad un torneo già esistente: - Il sistema chiede al gestore di inserire i dati validi - Si riprende dal passo 2 4a. Il gestore non conferma la creazione del torneo: - Il sistema annulla la procedura e ritorna al menù principale
Requisiti Speciali	Inserimento di regole di business nel passo 2
Elenco delle varianti tecnologiche e dei dati	Dati gestiti: - Dati relativi al torneo Persistenza:

	- I dati devono essere gestiti in memoria tramite strutture dati java - La persistenza è garantita tramite file di testo .txt
Frequenza di ripetizione	Sporadica
Varie	

UC5: CREAZIONE SQUADRA

Nome del caso d'uso	Creazione Squadra
Portata	Applicazione MatchUp
Livello	Obbiettivo Utente
Attore Primario	Giocatore
Parti interessate	Giocatore: Vuole creare una squadra con la quale partecipare al torneo.
Pre-condizioni	- Tutti i giocatori che faranno parte della squadra devono prima essere registrati sul sistema. - Il giocatore che crea la squadra ha effettuato il login con successo
Garanzia di successo	Il Giocatore (colui che ha creato la squadra) registra la propria squadra nel sistema.
Scenario principale di successo	1. Il giocatore seleziona "Crea Squadra". 2. Il giocatore inserisce il nome della squadra e la tipologia della squadra. 3. Il giocatore inserisce il numero di membri che compongono la squadra. 4. Il giocatore inserisce l'username del membro che vuole farne parte. Il passo 3 viene ripetuto finché serve. 5. Il sistema una volta ultimato l'inserimento giocatori mostra il riepilogo squadra. 6. Il giocatore conferma la registrazione della propria squadra. 7. Il sistema salva la squadra e la rende

	disponibile per l'iscrizione ai vari tornei.
Estensioni	3a. Il Giocatore inserisce il nome di una squadra già esistente: - Il sistema mostra l'errore all'utente e richiede di inserire un nome diverso 6a . Il Giocatore inserisce un username non valido o non presente nel sistema. - Il sistema segnala l'errore e richiede l'inserimento. 7a . Il giocatore non conferma registrazione squadra. - Il sistema ritorna al menù.
Requisiti Speciali	Inserimento di regole di business nel passo 2
Elenco delle varianti tecnologiche e dei dati	Dati gestiti: • Dati dei giocatori • Dati della squadra Persistenza: • Gestione dei dati in memoria tramite strutture dati Java (es. Liste, Map). Persistenza su file di testo (.txt) per il salvataggio delle squadre
Frequenza di ripetizione	Diverse volte al giorno
Varie	

UC6: ISCRIZIONE TORNEO

Nome del caso d'uso	Iscrizione Torneo
Portata	Applicazione MatchUp
Livello	Obbiettivo Utente
Attore Primario	Giocatore
Parti interessate	- Gestore della struttura: vuole registrare tutte le prenotazioni dei giocatori al torneo che si svolge nella sua struttura. - Giocatore : desidera iscrivere la propria squadra al torneo.
Pre-condizioni	- Il torneo deve essere stato già creato in precedenza e presente nel sistema - Il giocatore deve aver effettuato l'accesso su MatchUp
Garanzia di successo	Il giocatore si iscrive e partecipa con la propria squadra al torneo.

Scenario principale di successo	1. Il Giocatore seleziona "Iscrizione ad un Torneo". 2. Il giocatore cerca, in base alla tipologia di Torneo desiderato ,il Torneo a cui iscrivere la propria squadra. 3. Il giocatore inserisce il nome della propria squadra con cui intende partecipare al torneo 4. Il Sistema mostra il riepilogo dell'iscrizione con la relativa squadra partecipante e l'importo dovuto. 5. Il giocatore conferma l'iscrizione e l'importo dovuto viene scalato dal proprio conto personale.
---------------------------------	---

Estensioni	3a. Il Giocatore inserisce il nome di una squadra già iscritta al torneo: - Il sistema mostra l'errore all'utente e richiede di inserire un nome diverso 5a . Il giocatore non conferma l'iscrizione: - Il sistema ritorna al menù 5b. Il Giocatore non ha abbastanza credito: - Il sistema mostra l'errore e invita alla ricarica del conto. - Il sistema ritorna al menù 5c. Non vengo superati i controlli sull'iscrizione (iscrizioni chiuse, giocatore già iscritto al torneo con un'altra squadra): - Il sistema mostra l'errore e ritorna al menù
Requisiti Speciali	
Elenco delle varianti tecnologiche e dei dati	Dati gestiti: • Dati relativi alle iscrizioni Persistenza: • Gestione dei dati in memoria tramite strutture dati Java (es. Liste, Map). Persistenza su file di testo (.txt) per il salvataggio delle iscrizioni
Frequenza di ripetizione	Da sporadica a diverse volte al giorno
Varie	

UC7: GENERA CALENDARIO TORNEO

Nome del caso d'uso	Genera calendario torneo
Portata	Applicazione MatchUp
Livello	Obbiettivo Utente
Attore Primario	Gestore della struttura
Parti interessate	- Gestore della struttura: vuole generare il calendario delle partite del torneo che si svolge nella sua struttura - Giocatore : Vuole conoscere le date delle partite da giocare
Pre-condizioni	- Il gestore deve essere loggato - La struttura che ospita il torneo deve essere registrata su MatchUp e le iscrizioni del torneo devono essere chiuse
Garanzia di successo	Il sistema genera il calendario delle partite sulla base delle date libere e i campi dove avvengono le partite del torneo vengono riservati
Scenario principale di successo	1. Il gestore seleziona “genera calendario torneo”. 2. Il gestore inserisce il codice del torneo per il quale generare il calendario. 3. Il sistema crea un calendario sulla base delle squadre iscritte e delle date disponibili dei campi dove si svolgeranno le partite del torneo. 4. Il sistema restituisce al gestore la lista delle squadre con i partecipanti e le date delle partite.

Estensioni	3a. Il gestore inserisce il codice di un torneo non corretto o non esistente: - Il sistema mostra l'errore - Il gestore inserisce il codice corretto del torneo 3b. Il gestore inserisce il codice di un torneo concluso: - Il sistema segnala l'errore e mostra la classifica del torneo concluso 4a. Non si raggiunge il numero massimo di squadre per il torneo: - Il sistema segnala errore e annulla il torneo per il quale sta generando il calendario
Requisiti Speciali	
Elenco delle varianti tecnologiche e dei dati	Dati gestiti: • Dati in Input: Codice identificativo del torneo, elenco delle squadre iscritte, disponibilità oraria e date dei campi della struttura. • Dati in Output: Calendario delle partite generato (accoppiamento squadre, data, ora) stampato a schermo in console. Persistenza dati : Dati caricati da file di testo (.txt) all'avvio dell'applicazione. La gestione dei dati elaborati avviene tramite strutture dati Java (come LinkedList o HashMap).
Frequenza di ripetizione	Sporadica
Varie	

UC8: ASSEGNAZIONE ESITO PARTITA

Nome del caso d'uso	Assegnazione esito partita
Portata	Applicazione MatchUp
Livello	Obbiettivo Utente
Attore Primario	Gestore della struttura
Parti interessate	- Gestore della struttura: Vuole definire l'esito della partita del torneo e assegnare i punti alle squadre - Squadra(insieme di giocatori): Vuole conoscere i punti ottenuti

Pre-condizioni	- Il gestore deve essere loggato -Esiste un torneo in corso di svolgimento con almeno una partita disputata
Garanzia di successo	Il sistema registra il risultato della partita e assegna i punti correttamente alle squadre che hanno partecipato alla partita.

Scenario principale di successo	1. Il gestore seleziona “assegna esito partita” 2. Il gestore inserisce il codice del torneo di cui la partita fa parte 3. Il gestore immette l'id della partita conclusa 4. Il gestore assegna l'esito della partita (vittoria per una delle due squadre oppure pareggio). 5. Il sistema assegna i punti alla squadra e aggiorna la classifica del torneo
Estensioni	2a. Il gestore inserisce il codice di un torneo non esistente o concluso: - Il sistema mostra l'errore e ritorna al menu principale 3a. Il gestore inserisce l'id di una partita non valido: - Il sistema segnala l'errore e richiede l'inserimento dell'id corretto 3b. Il gestore inserisce l'id di una partita non ancora disputata: - Il sistema mostra l'errore e chiede l'inserimento dell'id corretto
Requisiti Speciali	Inserimento delle regole di dominio nel punto 5
Elenco delle varianti tecnologiche e dei dati	Dati gestiti: • Dati in Input: Codice identificativo del torneo, ID della partita conclusa, esito della partita selezionato dal gestore • Dati in Output: classifica del torneo aggiornata. Persistenza: Dati relativi a tornei, squadre e partite caricati all'avvio da file di testo (.txt). L'aggiornamento dello stato della partita e il ricalcolo dei punti in classifica vengono gestiti tramite strutture dati Java (come LinkedList o HashMap).

Frequenza di ripetizione	Frequente durante lo svolgimento del torneo
Varie	

UC9: ANNULLAMENTO PRENOTAZIONE

Nome del caso d'uso	Annullamento prenotazione
Portata	Applicazione MatchUp
Livello	Obiettivo Utente
Attore Primario	Giocatore
Parti interessate	- Giocatore: Vuole poter annullare una prenotazione esistente
Pre-condizioni	- Il giocatore deve essere loggato sul sistema MatchUp
Garanzia di successo	La prenotazione viene annullata e la fascia oraria che era occupata da tale prenotazione diventa di nuovo disponibile
Scenario principale di successo	1. Il giocatore seleziona “annulla prenotazione 2. Il sistema mostra le prenotazioni in corso al giocatore. 3. Il giocatore immette l'id della prenotazione che vuole annullare 4. Il sistema calcola il rimborso da restituire e mostra il riepilogo dell'operazione compreso il rimborso calcolato 5. Il giocatore conferma l'annullamento 6. Il sistema annulla la prenotazione, la quale assume lo stato di annullata
Estensioni	2a. Il giocatore non ha nessuna prenotazione in corso: - Il sistema mostra il messaggio “non ci sono prenotazioni in corso” al giocatore e ritorna al menu principale 3a. Il giocatore immette un id di una prenotazione già conclusa: - Il sistema segnala l'errore al giocatore - Il sistema invita il giocatore a immettere un id valido. 5a. Il Giocatore non conferma l'annullamento della prenotazione: - Il sistema annulla l'operazione e ritorna al

	menu principale
Requisiti Speciali	Inserimento delle regole di dominio nel punto 4
Elenco delle varianti tecnologiche e dei dati	Dati gestiti: • Identificativo univoco della prenotazione. Dati calcolati: • Importo del rimborso spettante (calcolato secondo le regole di dominio). • Stato aggiornato della prenotazione (cambiamento da "in corso" ad "annullata"). • Fascia oraria del campo di gioco ripristinata allo stato disponibile. Visualizzazione delle prenotazioni in corso tramite l'uso di una lista.
Frequenza di ripetizione	Varie volte al giorno
Varie	

UC10: EFFETTUA RICARICA CONTO

Nome del caso d'uso	Effettua ricarica conto
Portata	Applicazione MatchUp
Livello	Obiettivo Utente
Attore Primario	Giocatore
Parti interessate	- Giocatore: Vuole poter ricaricare il proprio saldo per effettuare le prenotazioni o le iscrizioni
Pre-condizioni	- Il giocatore deve essere loggato sul sistema MatchUp
Garanzia di successo	Il saldo viene correttamente ricaricato della cifra indicata dal giocatore
Scenario principale di successo	1. Il giocatore seleziona effettua ricarica conto 2. Il giocatore inserisce la cifra che intende ricaricare 3. Il giocatore inserisce i dati personali della propria carta 4. Il giocatore conferma l'operazione 5. Il sistema effettua la ricarica al giocatore

Estensioni	2a. Il giocatore richiede la ricarica di una cifra superiore o inferiore agli importi consentiti: - Il sistema mostra il messaggio di errore al giocatore e ritorna al menu principale 3a. Il giocatore immette dei dati della carta non validi: - Il sistema segnala l'errore al giocatore - Il sistema invita il giocatore a immettere i dati validi. 4a. Il giocatore non conferma l'operazione di ricarica: - Il sistema annulla l'operazione e ritorna al menu principale
Requisiti Speciali	Inserimento delle regole di dominio nel punto 2
Elenco delle varianti tecnologiche e dei dati	Dati gestiti: • Importo da ricaricare • Dati strumento di pagamento: Numero di carta (16 cifre), Data di scadenza (MM/AA), CVV/CVC (3 cifre), Nome intestatario. Dati generati: • Saldo aggiornato del giocatore
Frequenza di ripetizione	Varie volte al giorno
Varie	

UC11: EFFETTUA RICARICA CONTO GIOCATORE

Nome del caso d'uso	Effettua ricarica conto giocatore
Portata	Applicazione MatchUp
Livello	Obiettivo Utente
Attore Primario	Gestore della struttura

Parti interessate	- Gestore della struttura: Vuole poter ricaricare il saldo di un giocatore. - Giocatore: Vuole poter ricaricare il proprio saldo per effettuare le prenotazioni o le iscrizioni
Pre-condizioni	- Il gestore della struttura deve essere loggato sul sistema MatchUp
Garanzia di successo	Il saldo viene correttamente ricaricato dal gestore per conto del giocatore
Scenario principale di successo	1. Il gestore seleziona effettua ricarica conto giocatore 2. Il gestore inserisce l'username del giocatore 3. Il gestore inserisce l'importo da ricaricare per conto del giocatore 4. Il gestore conferma l'operazione 5. Il sistema effettua la ricarica e il saldo del giocatore viene incrementato della cifra richiesta.
Estensioni	2a. Il giocatore non viene trovato nel sistema: - Il sistema segnala l'errore "<i>Utente non registrato</i>". - Il gestore reinserisce l'username dell'giocatore. - Se l'esito è nuovamente negativo, il sistema ritorna al menu principale. 3a. Il giocatore chiede di ricaricare una cifra inferiore o superiore agli importi possibili: - Il sistema mostra il messaggio d'errore. - Il gestore informa il giocatore e annulla l'operazione. - Il sistema ritorna al menu principale. 4a. Il gestore non conferma l'operazione di ricarica: - Il sistema annulla l'operazione e ritorna al menu principale
Requisiti Speciali	Inserimento delle regole di dominio nel punto 3
Elenco delle varianti tecnologiche e dei dati	Dati gestiti: • Importo da ricaricare, username giocatore Dati generati: • Saldo aggiornato del giocatore

Frequenza di ripetizione	Varie volte al giorno
Varie	

1.5 DOCUMENTO DI VISIONE

Il documento di visione rappresenta un riepilogo generale dei requisiti e delle funzionalità dell'applicativo, nonché le opportunità di business ed economiche del progetto. Tale documento può essere consultato leggendo il documento chiamato "Documento di visione.pdf".

1.6 REGOLE DI BUSINESS

Si riporta di seguito il riepilogo delle regole di business implementate nell'applicativo, imposte per garantire un corretto funzionamento di quest'ultimo.

ID	Regola	Modificabilità	Sorgente
R1	Il costo totale della prenotazione è determinato dalla tariffa oraria del campo, con una maggiorazione del 15% se la data ricade nel weekend(Sabato o Domenica).	Alta	Politica dell'applicazione
R2	Il costo totale della prenotazione è scontato del 10% nelle fasce feriali (9-12, 14-16).	Alta	Politica dell'applicazione
R3	Le prenotazioni devono avere una durata pari a un'ora o a multipli di essa; non sono ammessi orari intermedi o frazionati."	Alta	Politica dell'applicazione
R4	Ogni 3 prenotazioni viene applicato uno sconto del 15%.	Alta	Politica dell'applicazione
R5	L'età minima di registrazione al servizio MatchUp per i Giocatori è fissata a 14	Bassa	Norme Europee relative all'utilizzo dei servizi digitali
R6	L'età minima di registrazione al servizio MatchUp per i Gestori di una struttura è fissata a 18 anni.	Bassa	Norme Europee relative all'utilizzo dei servizi digitali

R7	Lo username scelto non deve contenere spazi bianchi e deve essere univoco	Bassa	Politiche di sicurezza del sistema
R8	La password deve avere una lunghezza compresa tra 8 e 30 caratteri e deve includere almeno una lettera maiuscola (A-Z); Almeno una cifra numerica (0-9); Almeno un carattere speciale @ # \$ % ! ? _	Bassa	Politiche di sicurezza del sistema
R9	Il torneo per essere disputato deve avere almeno 6 squadre iscritte	Alta	Politica dell'applicazione
R10	La squadra vincitrice di una partita ottiene 3 punti mentre quella perdente ottiene 0 punti. In caso di pareggio verrà assegnato un punto a ciascuna delle due squadre	Bassa	Regolamento dei centri sportivi

R11	La data limite di scadenza delle iscrizioni deve essere antecedente di almeno 3 giorni rispetto alla data d'inizio del torneo, per consentire la formulazione e pubblicazione del calendario.	Alta	Politica dell'applicazione
R12	Un giocatore non può essere inserito in più squadre afferenti allo stesso torneo.	Bassa	Politica dei centri sportivi
R13	Il costo dell'iscrizione ad un torneo viene scalato dal credito di tutti i membri della squadra	Alta	Politica dell'applicazione
R14	L'importo minimo da poter ricaricare nel conto è di 10€	Alta	Politica dell'applicazione
R15	L'importo massimo da poter ricaricare nel conto è di 300€	Alta	Politica dell'applicazione
R16	L'annullamento della prenotazione con un preavviso pari o superiore a 24 ore rispetto alla data d'inizio dà diritto a un rimborso integrale (100% dell'importo).	Alta	Politica dell'applicazione
R17	L'annullamento della prenotazione effettuato con un preavviso compreso tra 12 e 24 ore rispetto alla data di inizio dà diritto a un rimborso parziale del 50% dell'importo versato.	Alta	Politica dell'applicazione

R18	L'annullamento della prenotazione effettuato con meno di 12 ore di preavviso dalla data di inizio non dà diritto ad alcun rimborso (0%).	Alta	Politica dell'applicazione
R19	Le partite verranno disputate a partire dalle ore 17:00, con giorni prioritari i fine settimana, ma qualora non ci fossero orari liberi, le partite si potranno svolgere nei giorni feriali	Alta	Politica dell'applicazione

1.7 SPECIFICHE SUPPLEMENTARI

Il software prodotto, oltre ai requisiti esposti in precedenza, terrà conto di questi ulteriori parametri supplementari, appartenenti alle seguenti aree:

- **Usabilità:** Il software non è dotato di interfaccia grafica, cercando di garantire un utilizzo semplice ed immediato, con indicazioni chiare e dirette su come interagire.
- **Affidabilità:** il software garantisce un'esecuzione corretta in un contesto mono-utente, coerentemente con la natura di applicazione a riga di comando sviluppata per questo progetto. Quindi al momento non è thread-safe. Un'eventuale evoluzione verso un contesto multiutente richiederebbe una revisione architetturale volta a eliminare lo stato condiviso a favore di un modello basato su sessioni utente indipendenti.
- **Vincoli hardware e software:** il software deve essere leggero e utilizzabile su una vasta gamma di calcolatori. L'implementazione tramite Java permette di avere una interoperabilità tra diversi sistemi operativi.

1.8 GLOSSARIO

Si riporta di seguito un breve glossario contenente i termini di uso comune nel dominio di interesse.

Termine	Definizione
Sistema	Applicativo MatchUp
Giocatore	Utente registrato che prenota i campi e partecipa ai tornei
Gestore	Persona titolare della struttura
Struttura	Insieme dei servizi messi a disposizione dal gestore.
Campo	Spazio destinato ad ospitare le partite all'interno

	della struttura
Partita	Evento che avviene tra due squadre in un torneo specifico
Torneo	Insieme di partite tra varie squadre dove la vittoria garantisce dei punti all'interno del torneo
Registrazione	Processo per cui il giocatore / struttura aderisce ai servizi offerti dal sistema
Username	Identificativo univoco di un utente per accedere al servizio
Password	Chiave segreta di un utente necessaria per accedere al servizio
Prenotazione	Documento che attesta il noleggio di un campo per una certa data e ora
Calendario	Insieme dei giorni di partite di uno specifico torneo
Squadra	Gruppo di giocatori assegnati che partecipano insieme al torneo
Iscrizione	Adesione di un giocatore alla partecipazione ad un torneo
Rimborso	Cifra che viene accreditata al saldo del giocatore in caso di annullamento della prenotazione

2. ANALISI

2.1 INTRODUZIONE

Lo sviluppo dell'applicazione è stato distribuito e caratterizzato da cinque iterazioni. Per ciascuna di esse si è effettuata un'analisi dei requisiti dove vengono scelti uno o più casi d'uso da implementare. Ad ogni iterazione inoltre viene effettuata, oltre all'implementazione di nuovi casi d'uso, anche il raffinamento di quelli precedenti se necessario. Per ciascuna iterazione sono stati considerati i seguenti requisiti:

Iterazione 1:

- Implementazione dello scenario di successo del caso d'uso UC3 relativo prenotazione di un campo in una determinata struttura e al calcolo del costo della prenotazione in accordo alle regole di dominio stabilite;
- Creazione di uno scenario di *setup*, ossia inserimento nel sistema di una serie di dati di test dell'applicativo.

Iterazione 2:

- Implementazione del caso d'uso UC1 relativo alla registrazione di un gestore e relativa struttura;
- Implementazione del caso d'uso UC2 relativo alla registrazione di un giocatore;

- Raffinamento del caso d'uso UC3 prenotazione campo che adesso terrà conto del numero delle prenotazioni già effettuate dal giocatore.

Iterazione 3:

- Implementazione del caso d'uso UC4 relativo alla creazione del torneo. Qui il gestore di una struttura vuole poter creare un nuovo torneo da ospitare all'interno della propria struttura;
- Implementazione del caso d'uso UC5 relativo alla creazione della squadra da poter iscrivere ai tornei pubblici. Qui i giocatori possono creare una squadra aggiungendo altri giocatori registrati sull'applicazione.
- Implementazione del caso d'uso UC6 relativo all'iscrizione ai tornei: Il giocatore, dopo aver creato la propria squadra, può procedere all'iscrizione di quest'ultima ad un torneo aperto sulla piattaforma;
- Implementazione del concetto di credito del giocatore, o meglio un conto ricaricabile che permette al giocatore di effettuare prenotazioni e iscrizioni e ovviamente ricevere rimborsi in caso di tornei e prenotazioni cancellate.

Iterazione 4:

- Implementazione del caso d'uso UC9 relativo all'annullamento delle prenotazioni. Questo caso d'uso permette di annullare una prenotazione tenendo conto delle politiche di rimborso definite nelle regole di dominio;
- Implementazione del caso d'uso UC10 relativo alla ricarica del conto del giocatore. Qui in questo caso d'uso si permette ai giocatori di poter ricaricare autonomamente il proprio conto, incrementando il saldo a disposizione;
- Implementazione del caso d'uso UC11. Anche qui permettiamo la ricarica del conto dei giocatori, ma non in autonomia, sarà il gestore di una struttura a poter ricaricare il conto.

Iterazione 5:

- Implementazione del caso d'uso UC7 relativo alla generazione dei calendari per i tornei. Qui permettiamo la generazione automatica del calendario delle partite, tenendo in considerazione le eventuali prenotazioni già esistenti di un determinato campo di una struttura.
- Implementazione del caso d'uso UC8 relativo all'assegnazione dell'esito alle partite. Qui si avrà la gestione dell'esito delle partite e quindi della classifica aggiornata del torneo. Il gestore al termine di una partita potrà registrare l'esito di quest'ultima e in base ai punteggi ottenuti dalle due squadre, verranno assegnati i punti e aggiornata la classifica.

2.2. MODELLO DI DOMINIO FINALE

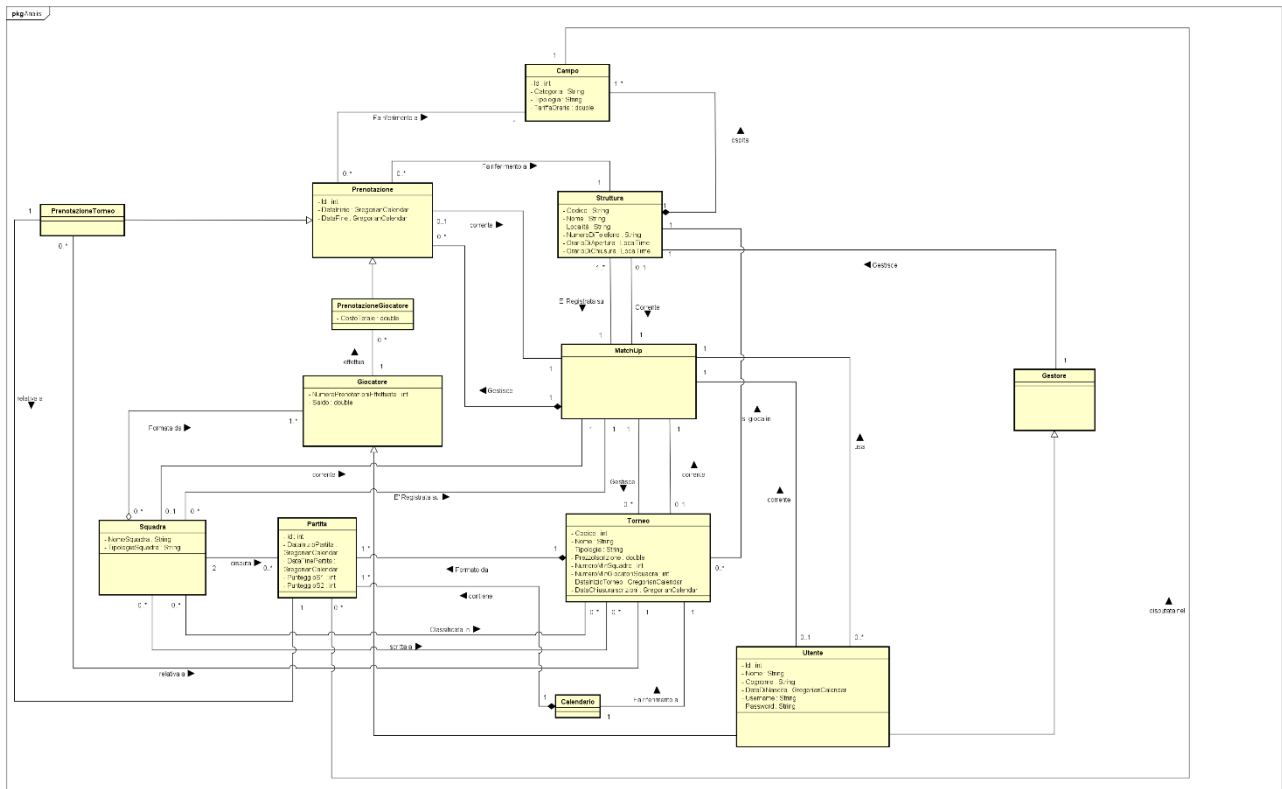


Figura 1: Modello di dominio

Qui si possono notare tutte le classi coinvolte nel dominio relativo all'applicazione, che sono le seguenti:

- MatchUp: rappresenta il SuD;
- Giocatore: persona che intende effettuare una prenotazione;
- Gestore: persona titolare di una *Struttura* registrato sul sistema;
- Struttura: impresa che possiede un certo numero di campi;
- Campo: oggetto che l'utente desidera prenotare per una fascia oraria limitata;
- Utente: Entità che può essere di tipo Giocatore o Gestore.
- Squadra: Entità composta da un gruppo di istanze Giocatore;
- Partita: Evento che avviene fra due istanze di squadra;
- Torneo: Entità composta da un gruppo di istanze Partita e gestito tramite un calendario;
- Prenotazione: oggetto base di *Prenotazione Giocatore* e *Prenotazione Torneo*;
- Prenotazione Giocatore: rappresenta la prenotazione effettuata dal Giocatore;
- Prenotazione Torneo: rappresenta la prenotazione effettuata automaticamente dal sistema per riservare un

campo per una partita del torneo

- Calendario: insieme delle partite di uno specifico torneo;

2.3 DIAGRAMMI DI SEQUENZA DI SISTEMA E CONTRATTI DELLE OPERAZIONI

2.3.1 DIAGRAMMA DI SEQUENZA DI SISTEMA UC1

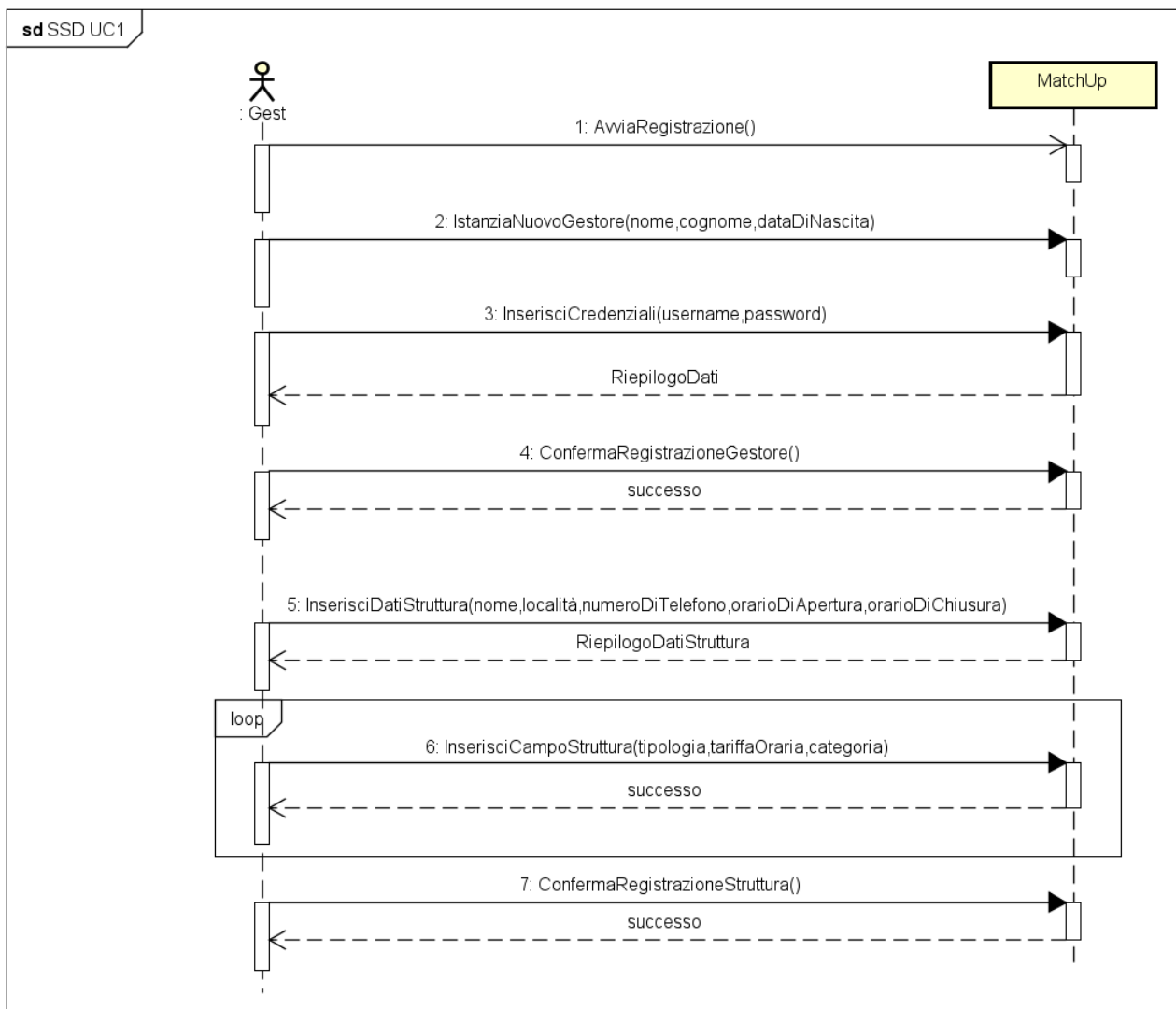


Figura 2: SSD UC1

2.3.1.1 ISTANZIA NUOVO GESTORE

Questa operazione permette al sistema di creare un'istanza di Gestore ge con i dati che quest'ultimo ha inserito.

Operazione	IstanzaNuovoGestore(nome:String,cognome:String, dataDiNascita:GregorianCalendar)
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)

Pre-condizioni	-
Post-condizioni	- E' stata creata un'istanza di Gestore ge - Gli attributi dell'istanza sono stati inizializzati (nome,cognome,dataDiNascita) - L'istanza ge è stata associata a MatchUp tramite l'associazione "corrente"

Tabella 1: Contratto dell'operazione Istanza nuovo gestore

2.3.1.2 INSERISCI CREDENZIALI

Questa operazione permette di associare le credenziali al gestore che si sta registrando

Operazione	InserisciCredenziali(username:String,password:String)
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)
Pre-condizioni	- E' in corso la registrazione di un Gestore ge
Post-condizioni	- Sono stati settati gli attributi username e password dell'istanza ge

Tabella 2: Contratto dell'operazione Inserisci credenziali

2.3.1.3 CONFERMA REGISTRAZIONE GESTORE

Questa operazione permette di confermare i dati inseriti dal gestore e di registrare l'istanza creata.

Operazione	ConfermaRegistrazioneGestore()
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)
Pre-condizioni	- L'istanza ge è stata creata
Post-condizioni	- L'istanza ge è stata associata a MatchUp tramite l'associazione "Usa"

Tabella 3: Contratto dell'operazione Conferma registrazione gestore.

2.3.1.4 INSERISCI DATI STRUTTURA

Questa operazione permette al gestore di inserire i dati della struttura da registrare sul sistema.

Operazione	InserisciDatiStruttura(nome:String,località:String, numeroTelefono: String, OrarioApertura:LocalTime, OrarioChiusura: LocalTime)
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)
Pre-condizioni	- Il gestore è stato registrato nel sistema
Post-condizioni	- E' stata creata un'istanza di struttura s - Gli attributi dell'istanza s sono stati inizializzati - L'istanza s è stata associata a MatchUp tramite l'associazione "corrente"

Tabella 4: Contratto dell'operazione Inserisci dati struttura

2.3.1.5 INSERISCI CAMPO STRUTTURA

Questa operazione permette al gestore di inserire i dati su un campo relativo ad una struttura da registrare sul sistema.

Operazione	InserisciCampoStruttura(categoriaCampo:String, tipologia:String,tariffaOraria:Double)
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)
Pre-condizioni	- E' in corso la registrazione di una struttura s
Post-condizioni	- E' stata creata un'istanza di campo c - Gli attributi dell'istanza c sono stati inizializzati - L'istanza c è stata associata alla struttura s tramite l'associazione "ospita"

Tabella 5: Contratto dell'operazione Inserisci campo struttura

2.3.1.6 CONFERMA REGISTRAZIONE STRUTTURA

Questa operazione permette di confermare i dati e di registrare la struttura nel sistema

Operazione	ConfermaRegistrazioneStruttura()
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)
Pre-condizioni	-Il gestore è stato registrato - E' in corso la registrazione di una struttura s
Post-condizioni	- L'istanza s è stata associata a MatchUp tramite l'associazione "E' registrata su" - L'istanza s è stata associata a Gestore ge tramite l'associazione "gestisce"

Tabella 6: Contratto dell'operazione Conferma registrazione struttura

2.3.2 DIAGRAMMA DI SEQUENZA DI SISTEMA UC2

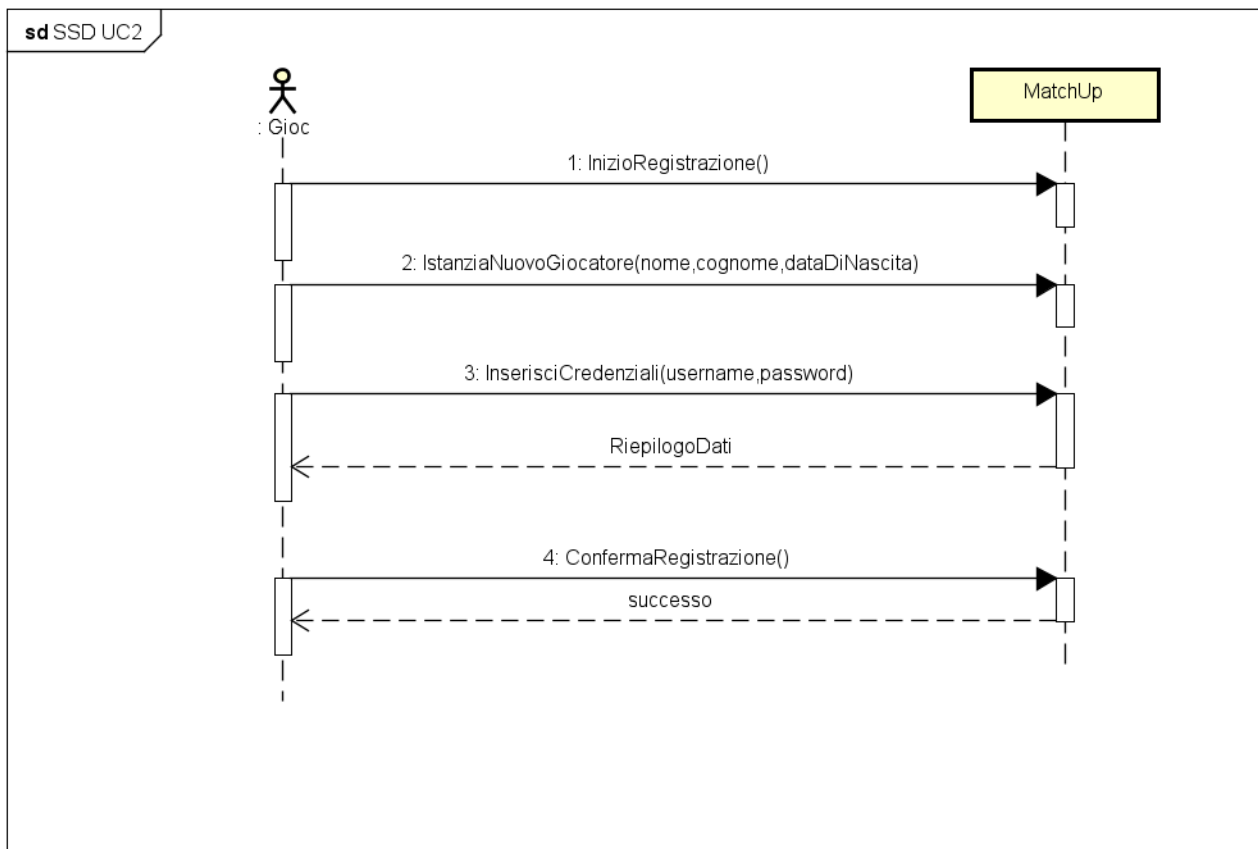


Figura 3: SSD UC2

2.3.2.1 ISTANZIA NUOVO GIOCATORE

Questa operazione permette al sistema di creare un'istanza di Giocatore g con i dati che quest'ultimo ha inserito.

Operazione	IstanziaNuovoGiocatore(nome:String,cognome:String, dataDiNascita:GregorianCalendar)
Riferimenti	Caso d'uso UC2 (Registrazione Giocatore)
Pre-condizioni	-
Post-condizioni	- E' stata creata un'istanza di Giocatore g - Gli attributi dell'istanza sono stati inizializzati (nome,cognome,dataDiNascita) - L'istanza g è stata associata a MatchUp tramite l'associazione "corrente"

Tabella 7: Contratto dell'operazione Istanza nuovo giocatore

2.3.2.2 INSERISCI CREDENZIALI

Questa operazione permette di associare le credenziali al giocatore che si sta registrando, verificando in particolare modo che lo username sia valido e non sia già in uso da parte di un altro giocatore e che la password sia valida.

Operazione	InserisciCredenziali(username:String,password:String)
Riferimenti	Caso d'uso UC2 (Registrazione Giocatore)
Pre-condizioni	- E' in corso la registrazione di un Giocatore g
Post-condizioni	- Sono stati settati gli attributi username e password dell'istanza g

Tabella 8: Contratto dell'operazione Inserisci credenziali

2.3.2.3 CONFERMA REGISTRAZIONE

Questa operazione permette di confermare i dati inseriti dal giocatore e di registrare l'istanza creata.

Operazione	ConfermaRegistrazione()
Riferimenti	Caso d'uso UC2 (Registrazione Giocatore)
Pre-condizioni	- L'istanza g è stata creata
Post-condizioni	- L'istanza g è stata associata a MatchUp tramite l'associazione "Usa"

Tabella 9: Contratto dell'operazione Conferma registrazione.

2.3.3 DIAGRAMMA DI SEQUENZA DI SISTEMA UC3

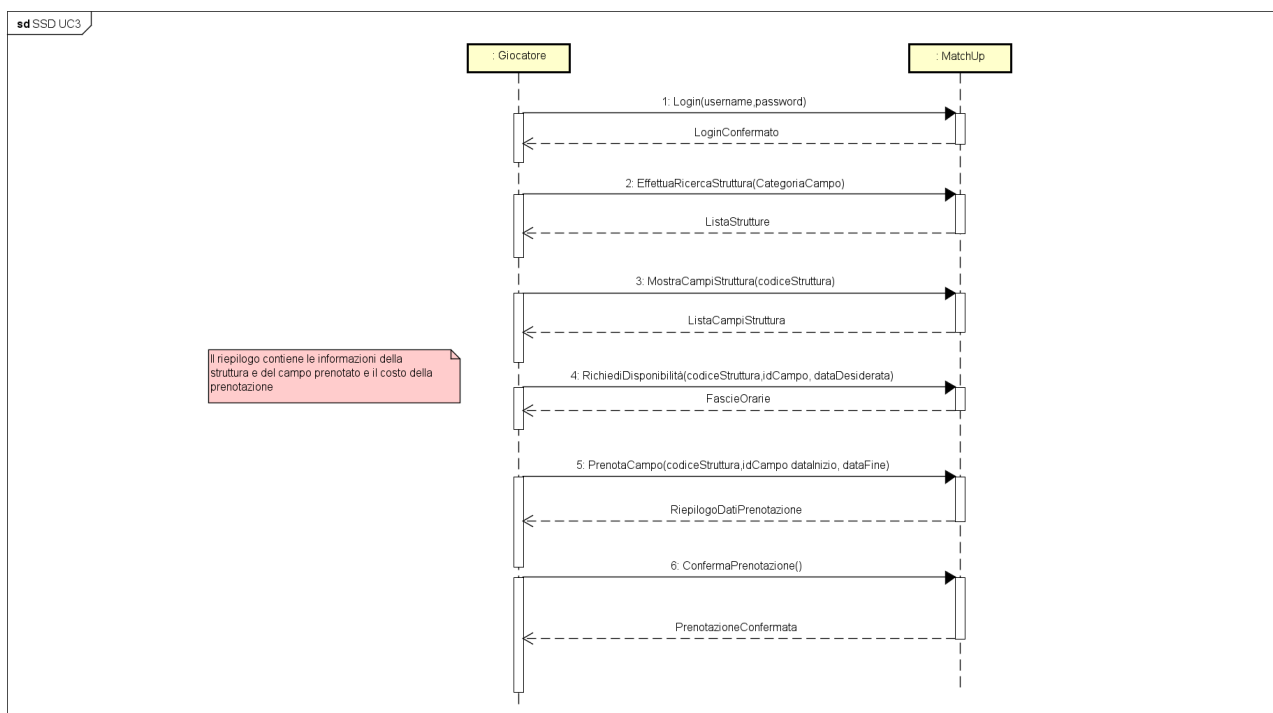


Figura 4: SSD UC3

2.3.3.1 LOGIN

Questa operazione permette a chi utilizza il sistema di autenticarsi prima di usufruire dei servizi (vale sia per i giocatori che per i gestori). Infatti per semplicità useremo l'istanza "utente u" per riferirci alle istanze di entrambe le classi coinvolte.

Operazione	Login(username:String , password:String)
Riferimenti	Caso d'uso UC3 (Effettua prenotazione)
Pre-condizioni	- L'utente (giocatore o gestore) devono essere registrati nel sistema
Post-condizioni	- L'istanza dell'utente u è stata associata a MatchUp tramite l'associazione "corrente"

Tabella 10: Contratto dell'operazione Login

2.3.3.2 EFFETTUA RICERCA STRUTTURA

Questa operazione permette, a partire dalla categoria del campo richiesto, di trovare e ritornare una lista di strutture che possiedono almeno un campo di quella categoria.

Operazione	EffettuaRicercaStruttura(categoriaCampo:String)
Riferimenti	Caso d'uso UC3 (EffettuaPrenotazione)
Pre-condizioni	- Sono state istanziate una o più strutture di test, ciascuna con i relativi campi . - Il parametro Categoria è un valore valido/non nullo.
Post-condizioni	- E' stata creata una lista di istanze di strutture idonee alla richiesta del giocatore

Tabella 11 Contratto operazione EffettuaRicercaStruttura

2.3.3.3 RICHIEDI DISPONIBILITA'

Questa operazione, fornita una data dal giocatore, permette di ritornare tutte le fasce orarie libere e non per prenotare il campo.

Operazione	RichiediDisponibilità(codiceStruttura:String,idCampo:String, dataDesiderata:GregorianCalendar)
Riferimenti	Caso d'uso UC3 (Effettua prenotazione)
Pre-condizioni	- E' in corso una prenotazione p - La struttura e il campo relativi alla prenotazione esistono
Post-condizioni	- Vengono ritornate al giocatore le fasce orarie in base alla data che ha inserito.

Tabella 12: Contratto dell'operazione Richiedi disponibilità

2.3.3.4 PRENOTA CAMPO

Questa operazione permette al giocatore di riservare il campo per l'ora e data richiesta.

Operazione	PrenotaCampo(CodiceStruttura:String,IdCampo:int,DataInizio:GregorianCalendar,DataFine:GregorianCalendar)
Riferimenti	Caso d'uso UC3 (EffettuaPrenotazione)
Pre-condizioni	- Il giocatore ha scelto il campo che intende prenotare per la data e l'ora richiesta
Post-condizioni	- E' stata creata un'istanza di prenotazione p - Gli attributi di p sono stati inizializzati - p è stata associata a Matchup tramite l'associazione corrente

Tabella 13 Contratto operazione PrenotaCampo

2.3.3.5 CONFERMA PRENOTAZIONE

Questa operazione permette di confermare i dati della prenotazione che verrà poi registrata nel sistema

Operazione	ConfermaPrenotazione(CodiceStruttura:String,IdCampo:int)
Riferimenti	Caso d'uso UC3 (EffettuaPrenotazione)
Pre-condizioni	- E' in corso una prenotazione p
Post-condizioni	- La prenotazione è stata associata a Matchup tramite l'associazione "Gestisce". - p è stata associata a campo tramite l'associazione "Fa riferimento a" - p è stata associata alla struttura s tramite l'associazione "Fa riferimento a" - p è stata associata al giocatore g1 tramite l'associazione "Effettua"

Tabella 14 Contratto operazione ConfermaPrenotazione

2.3.4 DIAGRAMMA DI SEQUENZA DI SISTEMA UC4

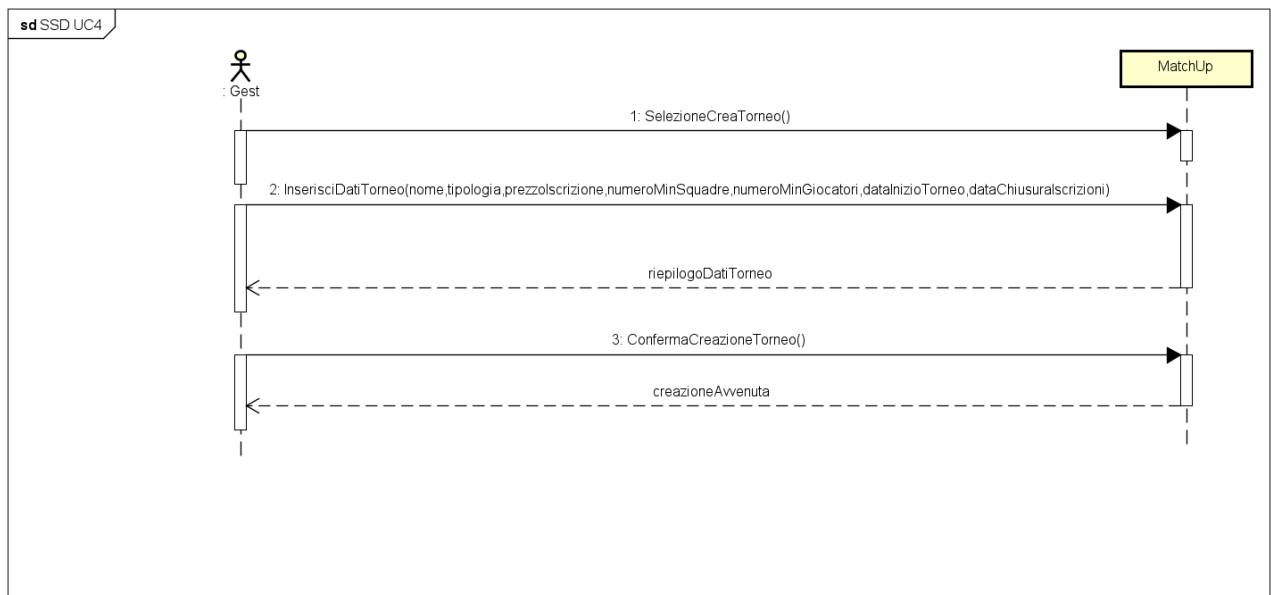


Figura 5: SSD UC4

2.3.4.1 INSERISCI DATI TORNEO

Questa operazione permette al gestore di inserire i dati del torneo da creare e registrare nel sistema.

Operazione

Riferimenti

Pre-condizioni

Post-condizioni

InserisciDatiTorneo (nome: String, tipologia: String, prezzoscrizione:double, numeroMinGiocatoriSquadra:int, dataInizioTorneo:GregorianCalendar, dataChiusuraScrizioni: GregorianCalendar)
Caso d'uso UC4 (Creazione nuovo torneo)
Il gestore è loggato sul sistema

- E' stata creata un'istanza di Torneo t
- Gli attributi dell'istanza sono stati inizializzati
- L'istanza t è stata associata alla struttura s tramite l'associazione "si gioca in"
- L'istanza t è stata associata a MatchUp tramite l'associazione "corrente"
- Viene mostrato il riepilogo dei dati del torneo

Tabella 15: Contratto dell'operazione Inserisci dati torneo

2.3.4.2 CONFERMA CREAZIONE TORNEO

Questa operazione permette di confermare i dati e registrare il torneo nel sistema, per renderlo disponibile alle iscrizioni.

Operazione	ConfermaCreazioneTorneo()
Riferimenti	Caso d'uso UC4 (Creazione nuovo torneo)
Pre-condizioni	L'istanza t di torneo è stata creata
Post-condizioni	L'istanza t è stata associata a MatchUp tramite l'associazione "gestisce"

Tabella 16: Contratto dell'operazione Conferma creazione torneo

2.3.5 DIAGRAMMA DI SEQUENZA DI SISTEMA UC5

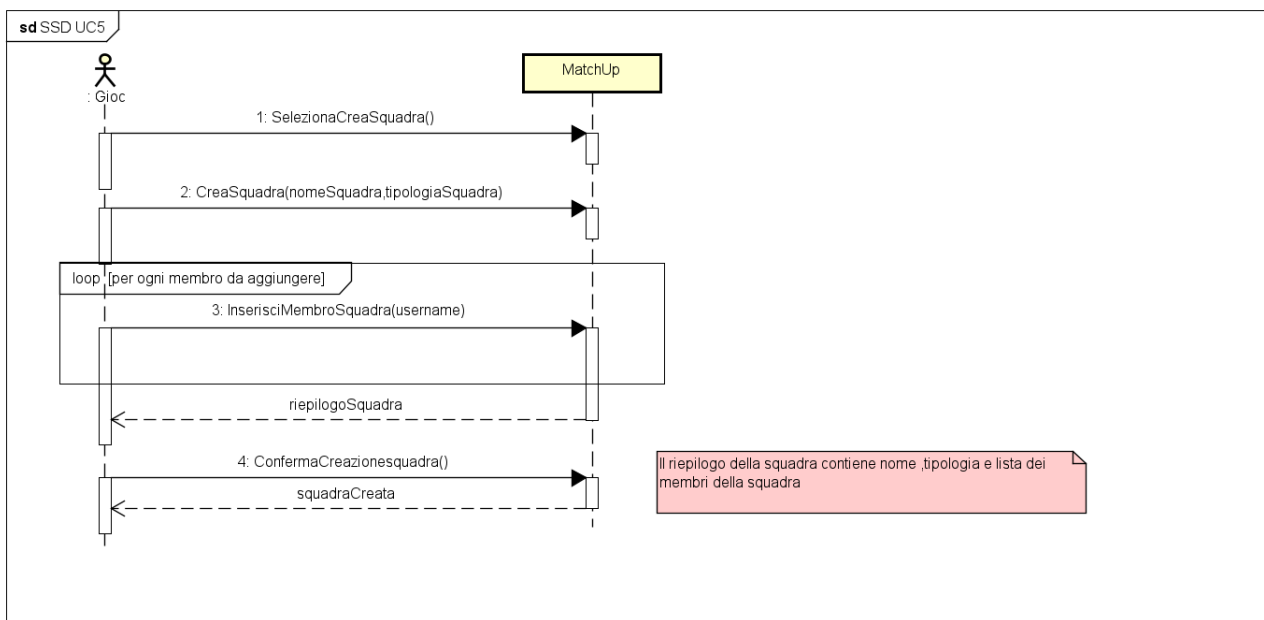


Figura 6: SSD UC5

2.3.5.1 CREA SQUADRA

Questa operazione permette al giocatore di creare la squadra da iscrivere al torneo

Operazione	CreaSquadra(nomeSquadra:String,tipologiaSquadra:String)
Riferimenti	Caso d'uso UC5 (Creazione squadra)

Pre-condizioni	Il giocatore che vuole creare la squadra è loggato su MatchUp
Post-condizioni	- E' stata creata un'istanza di Squadra sq - Gli attributi dell'istanza sono stati inizializzati - L'istanza sq è stata associata a MatchUp tramite l'associazione "corrente"

Tabella 17: Contratto dell'operazione Inserisci dati squadra

2.3.5.2 INSERISCI MEMBRO SQUADRA

Questa operazione permette al giocatore di aggiungere i membri alla squadra creata in precedenza

Operazione	InserisciMembroSquadra(username:String)
Riferimenti	Caso d'uso UC5 (Creazione squadra)
Pre-condizioni	- E' in corso la creazione dell'istanza sq - Esiste un'istanza di giocatore g con l'username esistente
Post-condizioni	- L'istanza giocatore g è associata alla squadra s tramite l'associazione "formata da"

Tabella 18: Contratto dell'operazione Inserisci membro squadra

2.3.5.3 CONFERMA CREAZIONE SQUADRA

Questa operazione permette di salvare i dati e registrare la squadra nel sistema.

Operazione	ConfermaCreazioneSquadra()
Riferimenti	Caso d'uso UC5 (Creazione squadra)
Pre-condizioni	- E' in corso la creazione dell'istanza sq
Post-condizioni	- L'istanza sq è associata a MatchUp tramite l'associazione "E' registrata su"

Tabella 19: Contratto dell'operazione Conferma creazione squadra

2.3.6 DIAGRAMMA DI SEQUENZA DI SISTEMA UC6

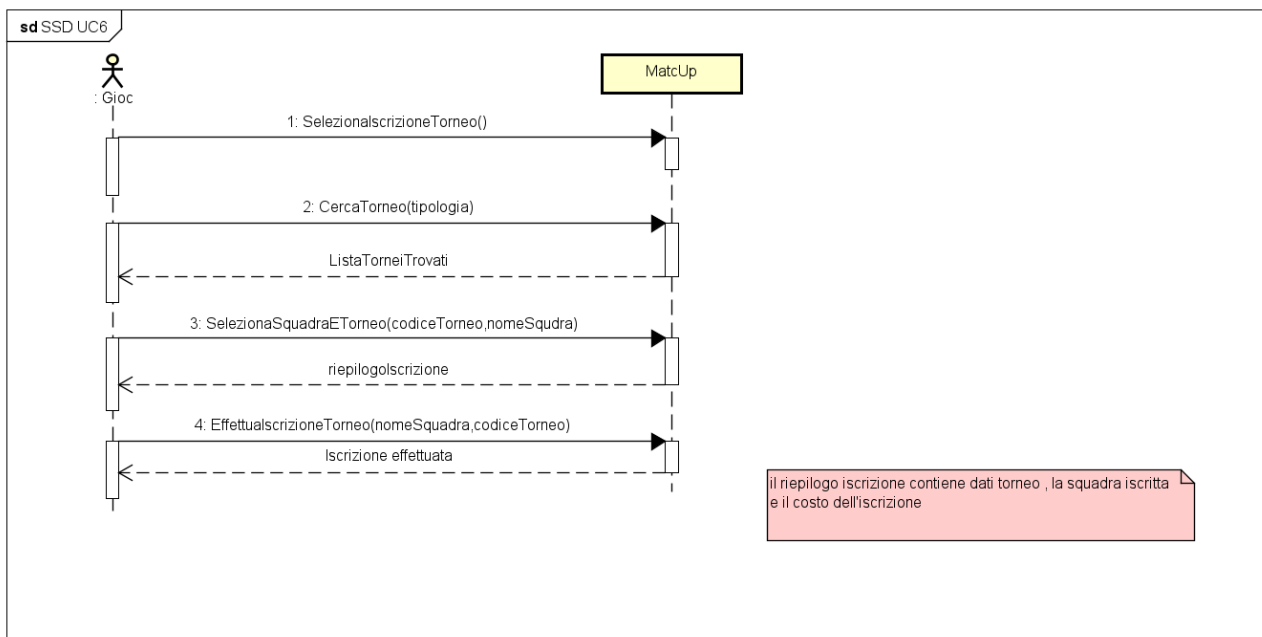


Figura 7: SSD UC6

2.3.6.1 CERCA TORNEO

Questa operazione permette al giocatore di vedere tutti i tornei disponibili in base alla tipologia che ha richiesto

Operazione	CercaTorneo(tipologia:String)
Riferimenti	Caso d'uso UC6(Iscrizione torneo)
Pre-condizioni	Il giocatore è loggato nel sistema
Post-condizioni	Il sistema ha ritornato la lista dei tornei trovati in base alla richiesta del giocatore

Tabella 20: Contratto dell'operazione Cerca torneo

2.3.6.2 SELEZIONA SQUADRA E TORNEO

Con questa operazione il giocatore sceglie il torneo e la squadra da iscrivere. L'operazione alla fine mostra un riepilogo prima dell'iscrizione finale.

Operazione	SelezionaSquadraETorneo(nomeSquadra:String,codiceTorneo:Int)
Riferimenti	Caso d'uso UC6(Iscrizione torneo)
Pre-condizioni	- E' stata creata l'istanza di Squadra sq - E' stata creata l'istanza di Torneo t
Post-condizioni	E' stato generato un riepilogo che mostra i dati della squadra che si sta iscrivendo e quelli del torneo

Tabella 21: Contratto dell'operazione Seleziona squadra e torneo

2.3.6.3 EFFETTUA ISCRIZIONE TORNEO

Questa operazione permette di effettuare l'iscrizione e registrarla sul sistema.

Operazione	EffettuaIscrizioneTorneo(nomeSquadra:String,codiceTorneo: Int)
Riferimenti	Caso d'uso UC6(Iscrizione torneo)
Pre-condizioni	- Il giocatore g è autenticato - Tutti i controlli sono stati validati
Post-condizioni	- L'istanza di Squadra sq è stata associata all'istanza torneo tramite l'associazione "iscritta a" - Il saldo dei giocatori della squadra è stato correttamente decrementato con il valore della quota di iscrizione.

Tabella 22: Contratto dell'operazione Effettua iscrizione torneo

2.3.7 DIAGRAMMA DI SEQUENZA DI SISTEMA UC7

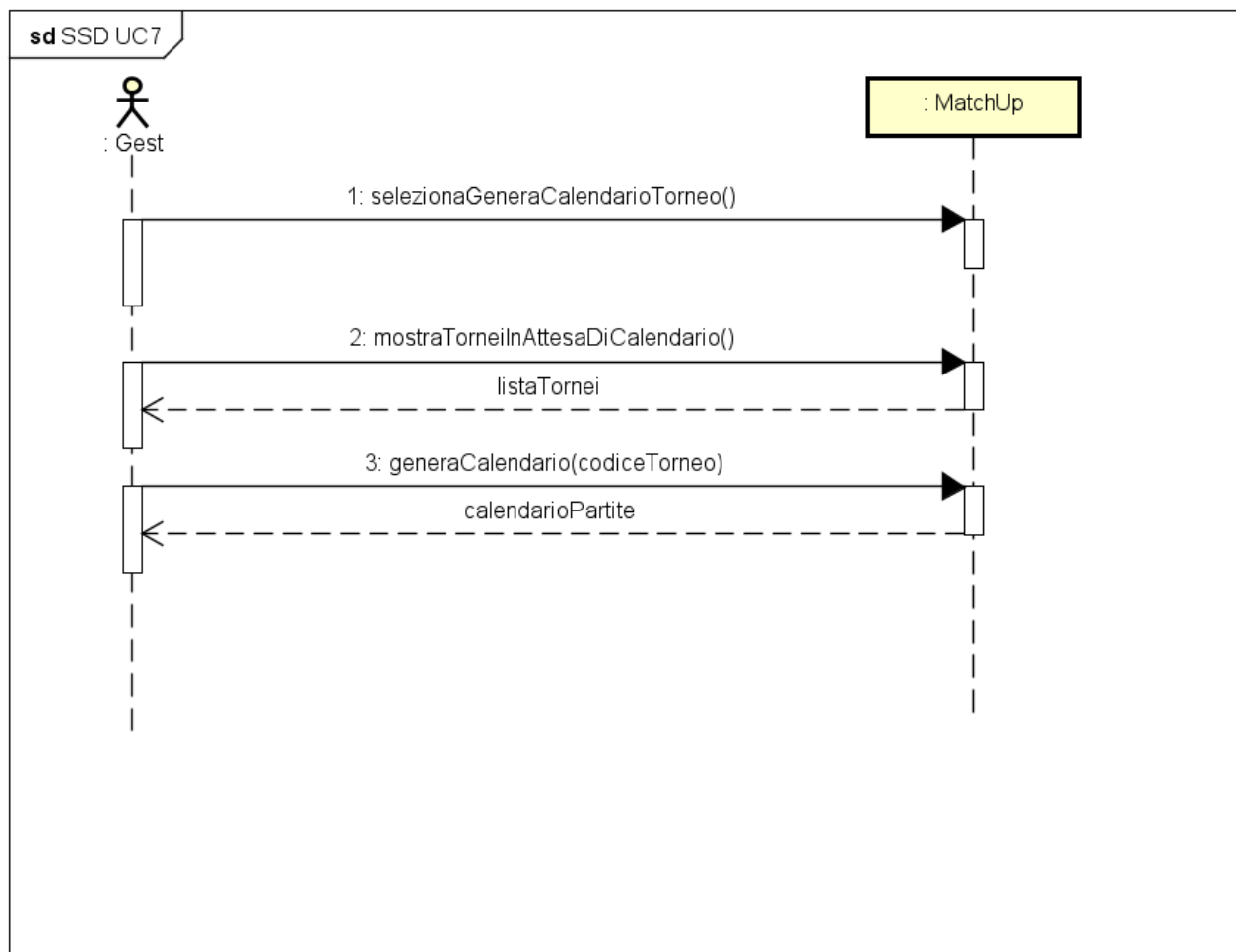


Figura 8: SSD UC7

2.3.7.1 GENERA CALENDARIO

Questa operazione permette al gestore di generare il calendario degli incontri per un determinato torneo.

Operazione	GeneraCalendario(codiceTorneo:_int)
Riferimenti	Caso d'uso UC7 (Genera calendario torneo)
Pre-condizioni	- È stata creata un'istanza di torneo <i>t</i>. - Il torneo <i>t</i> ha raggiunto il numero di squadre minimo - È trascorsa da data di scadenza della chiusura delle iscrizioni
Post-condizioni	- È stata creata un'istanza di Calendario <i>c</i> - L'istanza <i>c</i> viene associata al torneo <i>t</i> tramite l'associazione "fa riferimento a"

Tabella 23: contratto delle operazioni di Genera calendario

2.3.8 DIAGRAMMA DI SEQUENZA DI SISTEMA UC8

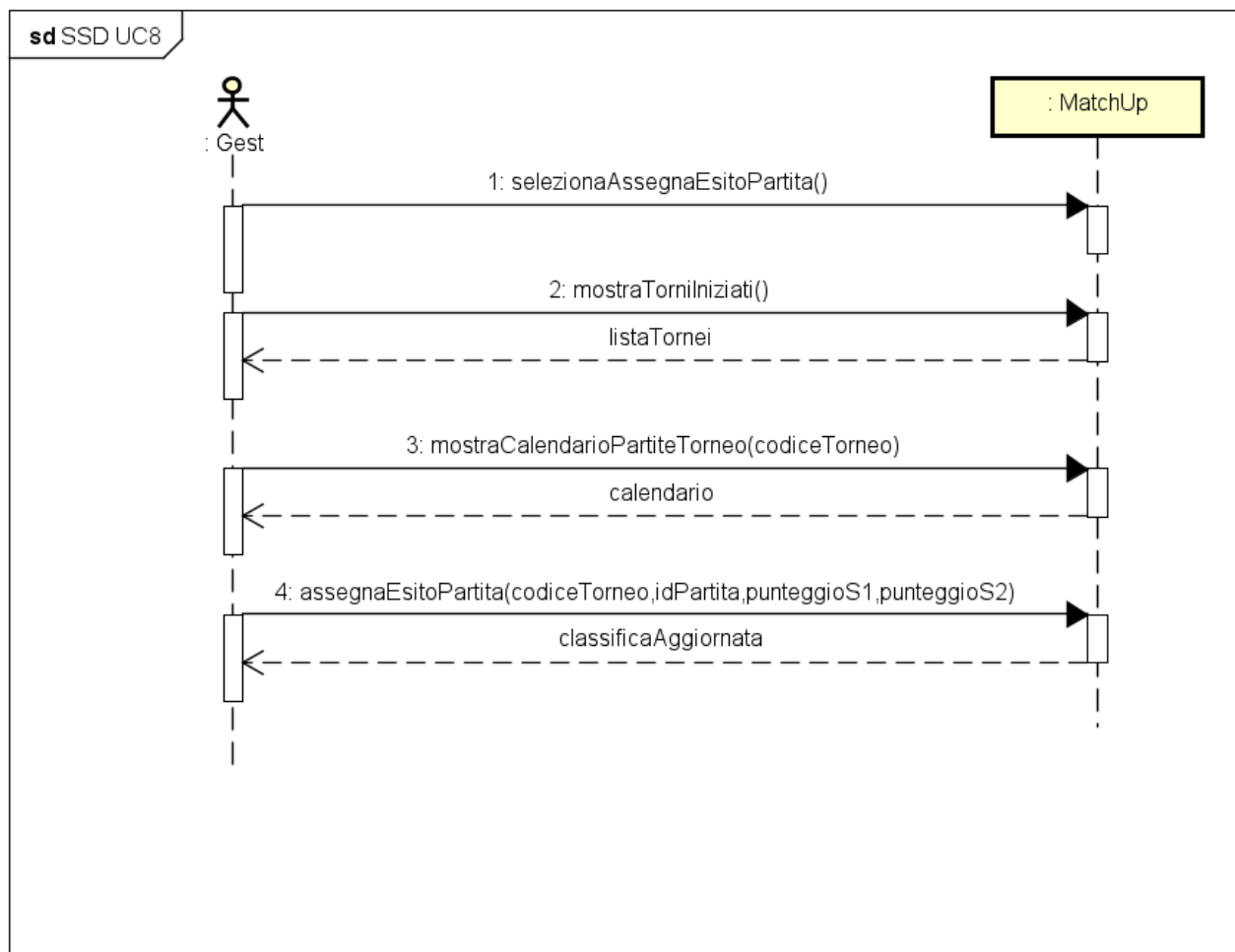


Figura 9: SSD UC8

2.3.8.1 ASSEGNA ESITO PARTITA

Questa operazione permette al gestore di determinare l'esito della partita e ottenere la classifica aggiornata con i punti assegnati correttamente alle squadre che hanno disputato quella partita

Operazione	AssegnaEsitoPartita(codiceTorneo:_int,idPartita:int,punteggioS1:int,punteggioS2:int)
Riferimenti	Caso d'uso UC8 (Assegnazione esito partita)
Pre-condizioni	- Il gestore è loggato su MatchUp - Esiste un'istanza del torneo <i>t</i> in corso - È stata disputata la partita <i>p</i> del torneo
Post-condizioni	- Sono stati settati gli attributi di <i>p</i> relativi ai punteggi delle due squadre. - Sono stati assegnati i punti alle squadre che hanno disputato la partita <i>p</i>

Tabella 24: contratto delle operazioni di Assegna esito partita

2.3.9 DIAGRAMMA DI SEQUENZA DI SISTEMA UC9

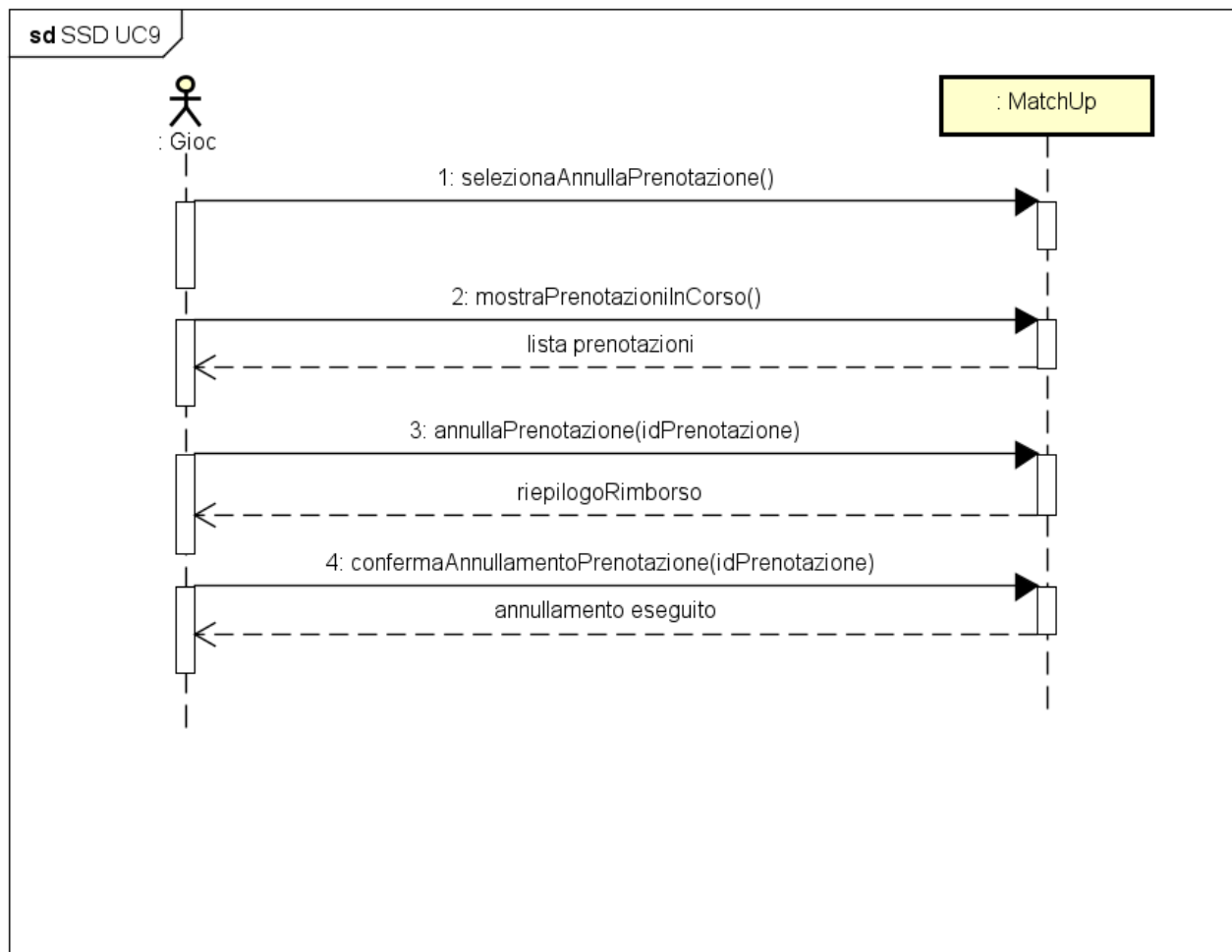


Figura 10: SSD UC9

2.3.9.1 ANNULLA PRENOTAZIONE

Questa operazione permette al giocatore di poter annullare le prenotazioni che ha effettuato in precedenza.

Operazione	AnnullaPrenotazione(idPrenotazione)
Riferimenti	Caso d'uso UC9 (Annullamento prenotazione)
Pre-condizioni	- Il giocatore g è loggato su MatchUp - Esiste un'istanza p di prenotazione associata al giocatore g
Post-condizioni	È stato calcolato l'importo del rimborso in base alle politiche di cancellazione del sistema e memorizzato temporaneamente per la conferma.

Tabella 25: Contratto dell'operazione Annulla prenotazione

2.3.9.2 CONFERMA ANNULLAMENTO PRENOTAZIONE

Questa operazione conferma l'annullamento della prenotazione del giocatore e permette il rimborso sul suo conto personale

Operazione	ConfermaAnnullamentoPrenotazione(idPrenotazione)
Riferimenti	Caso d'uso UC9 (Annullamento prenotazione)
Pre-condizioni	- Il giocatore <i>g</i> è loggato su MatchUp. - Esiste un'istanza <i>p</i> di Prenotazione associata al giocatore <i>g</i>
Post-condizioni	- L'attributo stato della prenotazione <i>p</i> è stata settato su "rimborsata" - L'attributo saldo del giocatore <i>g</i> viene incrementato del costo della prenotazione (rimborso) - E' stata settata libera la fascia oraria occupata dalla prenotazione del giocatore.

Tabella 26: Contratto dell'operazione Conferma annullamento prenotazione

2.3.10 DIAGRAMMA DI SEQUENZA DI SISTEMA UC10

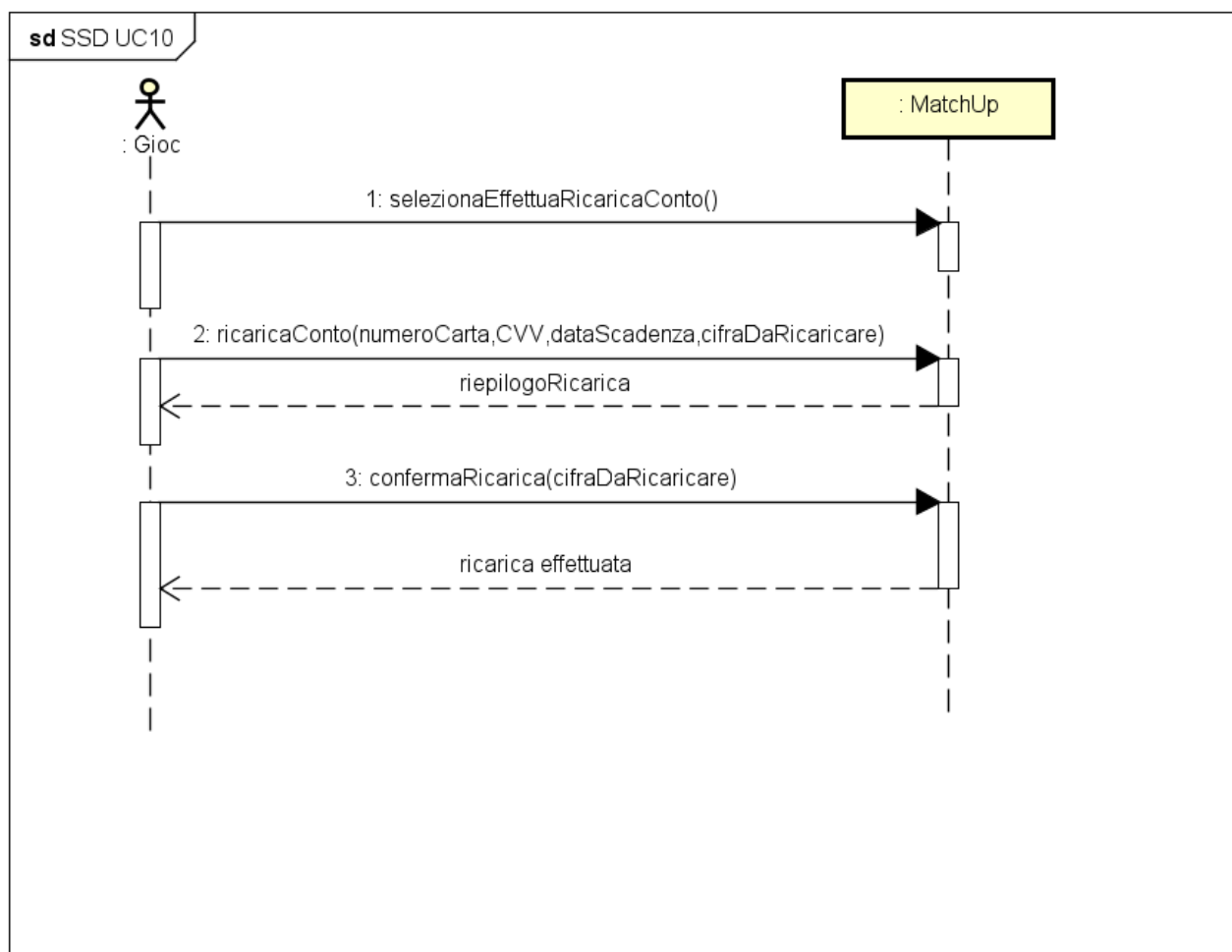


Figura 11: SSD UC10

2.3.10.1 RICARICA CONTO

Questa operazione permette al giocatore di ricaricare il proprio conto personale

Operazione	RicaricaConto(numeroCarta:String,CCV:Int,dataScadenza:GregorianCalendar,cifraDaRicaricare:double)
Riferimenti	Caso d'uso UC10 (Effettua ricarica conto)
Pre-condizioni	-Il giocatore g è loggato su MatchUp
Post-condizioni	Nessuna

Tabella 27: Contratto dell'operazione Ricarica conto

2.3.10.2 CONFERMA RICARICA

Questa operazione permette di conferma l'operazione di ricarica del conto

Operazione	ConfermaRicarica(numeroCarta:String,CCV:int,dataScadenza:GregorianCalendar,cifraDaRicaricare:double)
Riferimenti	Caso d'uso UC10 (Effettua ricarica conto)
Pre-condizioni	-Il giocatore g è loggato su MatchUp
Post-condizioni	- E' stato incrementato il valore dell'attributo saldo del giocatore g della quantità "cifraDaRicaricare"

Tabella 28: Contratto dell'operazione Conferma ricarica conto

2.3.11 DIAGRAMMA DI SEQUENZA DI SISTEMA UC11

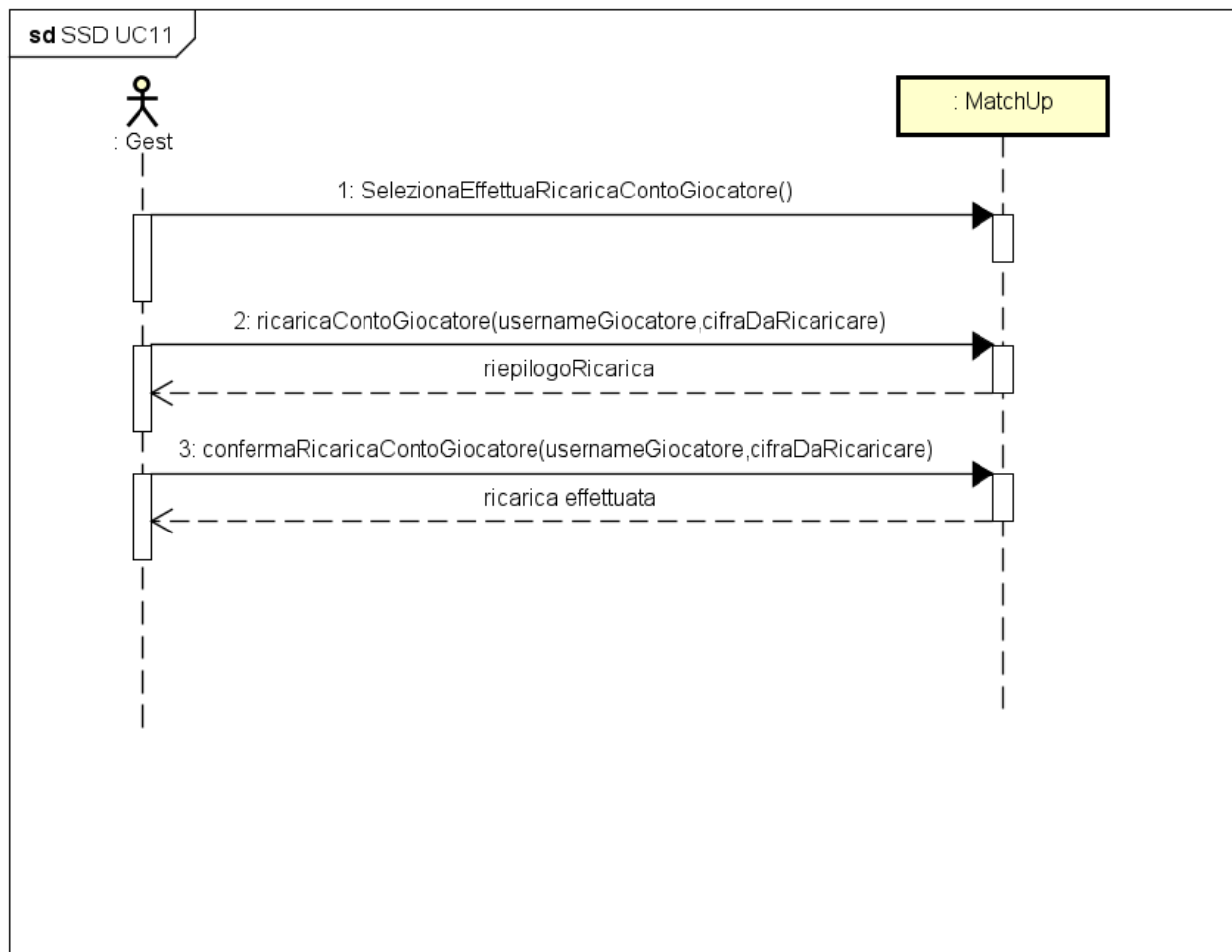


Figura 12: SSD UC11

2.3.11.1 RICARICA CONTO GIOCATORE

Questa operazione permette al gestore di accreditare sul conto del giocatore la cifra da egli richiesta.

Operazione	RicaricaContoGiocatore(username:String,cifraDaRicaricare:double)
Riferimenti	Caso d'uso UC11 (Effettua ricarica conto giocatore)
Pre-condizioni	-Il gestore ge è loggato su MatchUp -Il giocatore g è registrato su MatchUp
Post-condizioni	Nessuna

Tabella 29: Contratto dell'operazione Ricarica conto giocatore

2.3.11.2 CONFERMA RICARICA CONTO GIOCATORE

Questa operazione permette al gestore di confermare l'operazione di ricarica effettuata per conto del giocatore

Operazione	ConfermaRicaricaContoGiocatore(username:String,cifraDaRicaricare:double)
Riferimenti	Caso d'uso UC11 (Effettua ricarica conto giocatore)
Pre-condizioni	-Il gestore ge è loggato su MatchUp -Il giocatore g è registrato su MatchUp
Post-condizioni	- E' stato incrementato il valore dell'attributo saldo del giocatore g della quantità " cifraDaRicaricare "

Tabella 30: Contratto dell'operazione Conferma ricarica conto giocatore

3 PROGETTAZIONE

3.1 INTRODUZIONE

Nella fase di progettazione ci occupiamo di definire gli elaborati che servono per il processo di scrittura del codice. Questa fase si compone di due parti principali: la progettazione statica e la progettazione dinamica. La progettazione statica si concretizza nella realizzazione del Diagramma delle Classi di Progetto (DCD). Questo diagramma si sviluppa a partire dal modello di dominio definito nella fase di analisi, ma se ne differenzia in un aspetto fondamentale: mentre il modello di dominio rappresenta i concetti del mondo reale, le classi che compaiono nel DCD sono oggetti software veri e propri, progettati per avere una corrispondenza diretta con il codice sorgente dell'applicazione. La progettazione dinamica invece descrive il comportamento del sistema attraverso i diagrammi di sequenza, utili per comprendere come gli oggetti del dominio applicativo interagiscono tra loro al fine di realizzare i casi d'uso identificati nella fase di analisi.

Di seguito vengono presentati i diagrammi più significativi del progetto. Per una migliore consultazione, tutti i diagrammi sono disponibili in formato PNG nella cartella "diagrammi finali". Per la visione completa dei diagrammi di sequenza relativi alle singole iterazioni, è invece possibile consultare le rispettive cartelle "diagrammi", contenenti i file PNG di tutti i diagrammi di sequenza prodotti per ciascuna iterazione del progetto.

3.2.3 CONFERMA PRENOTAZIONE

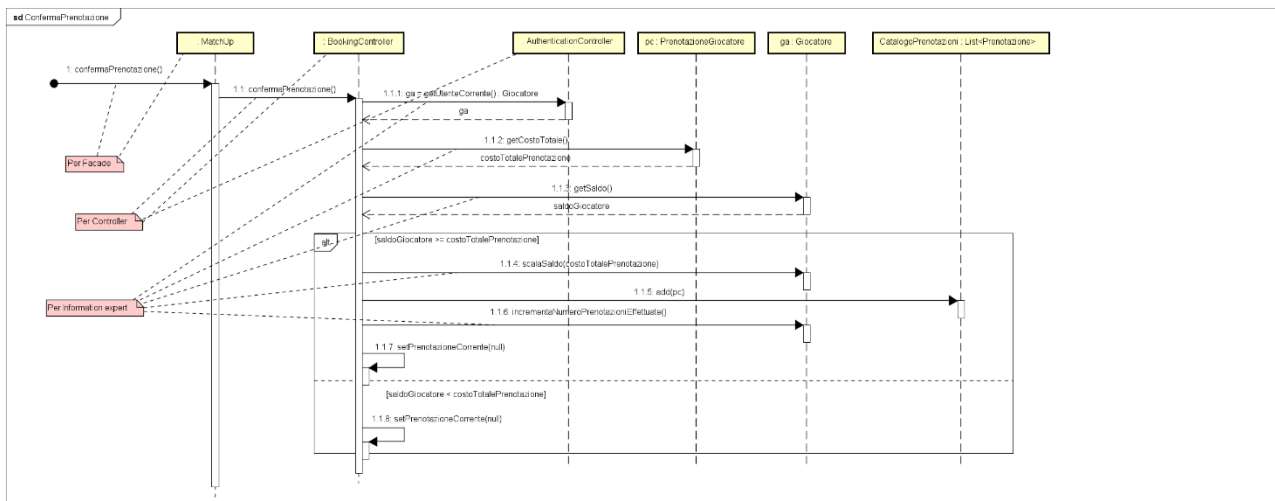


Figura 15: Diagramma di sequenza di conferma prenotazione

3.2.4 INSERISCI DATI TORNEO

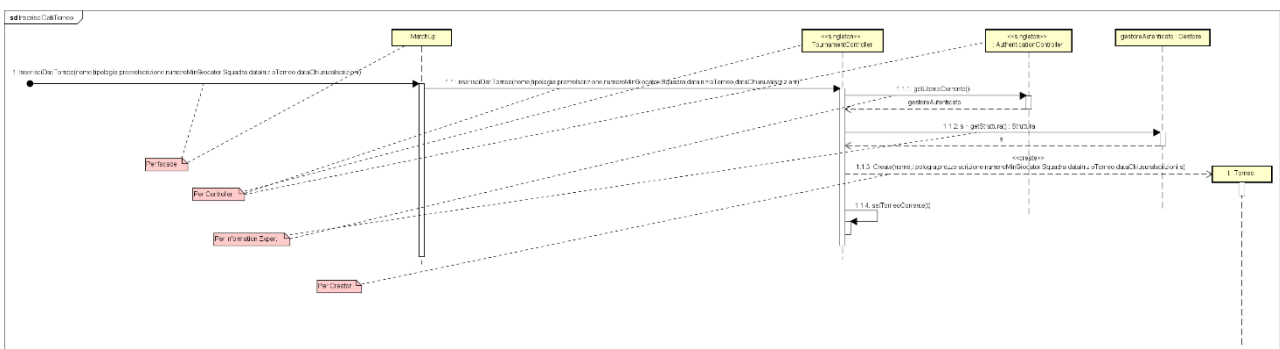


Figura 16: Diagramma di sequenza di inserisci dati torneo

3.2.5 CREA SQUADRA

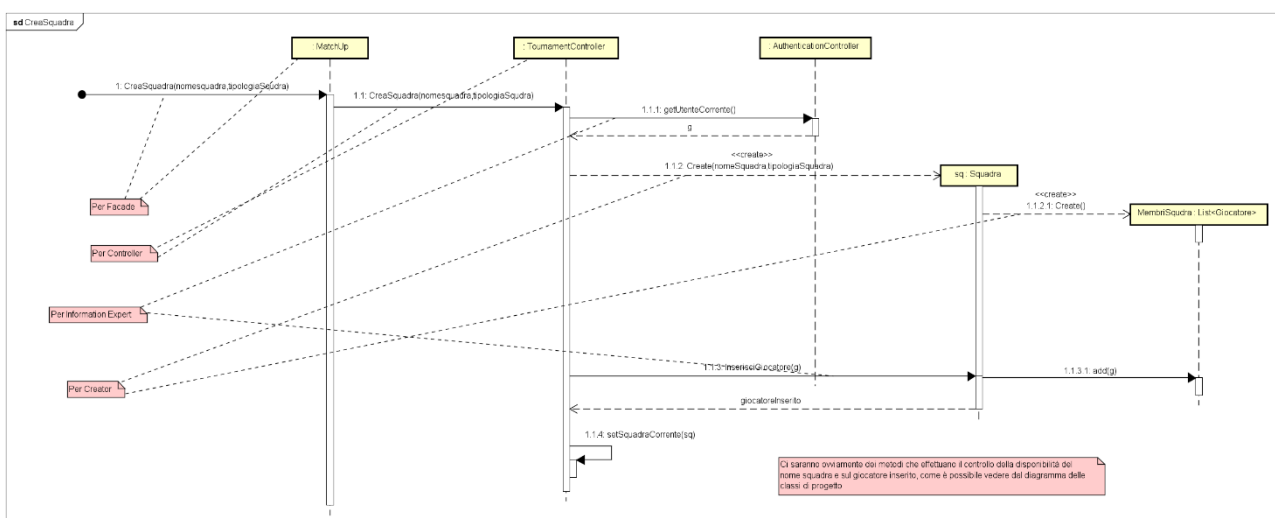


Figura 17: Diagramma di sequenza di crea squadra

3.2.8 GENERA CALENDARIO

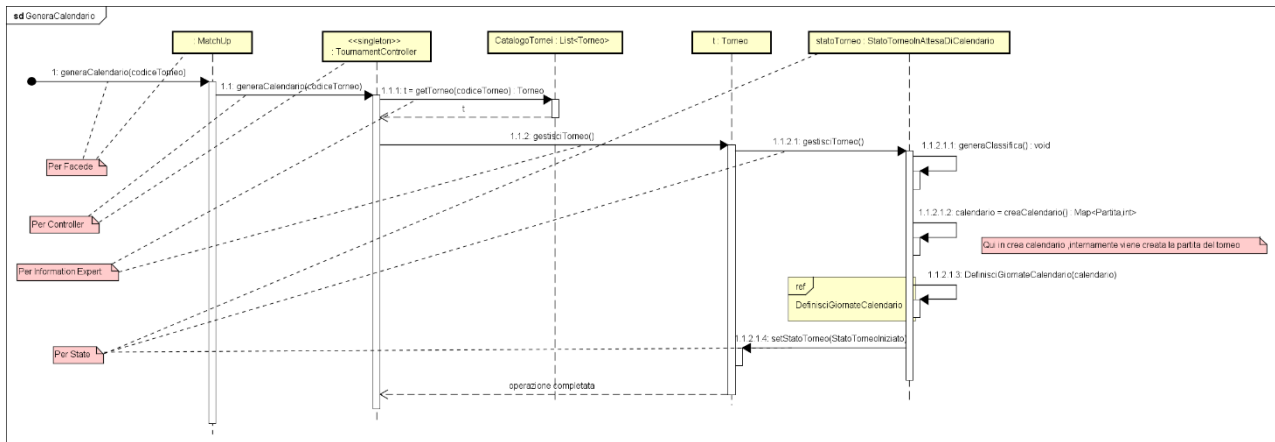


Figura 20: Diagramma di sequenza di genera calendario

3.2.9 DEFINISCI GIORNATE CALENDARIO

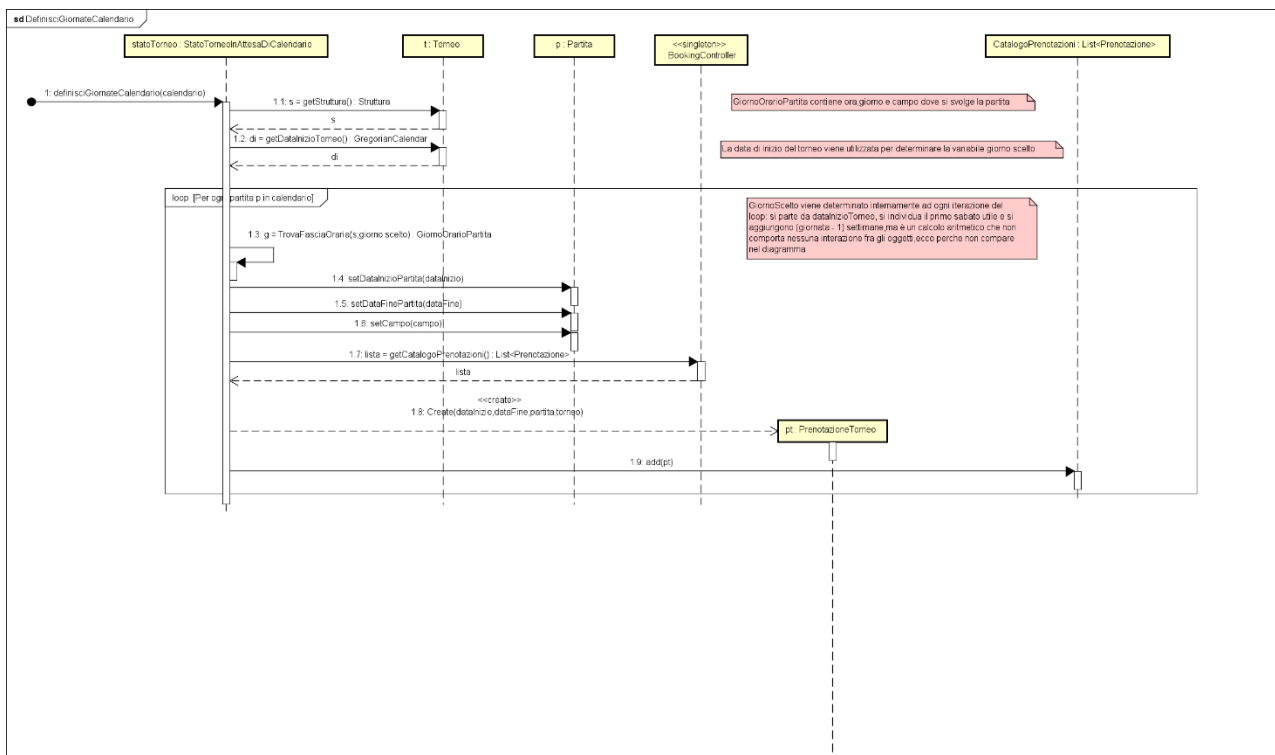


Figura 21: Diagramma di sequenza di definisci giornate calendario

3.2.10 ASSEGNA ESITO PARTITA E AGGIORNAMENTO CLASSIFICA

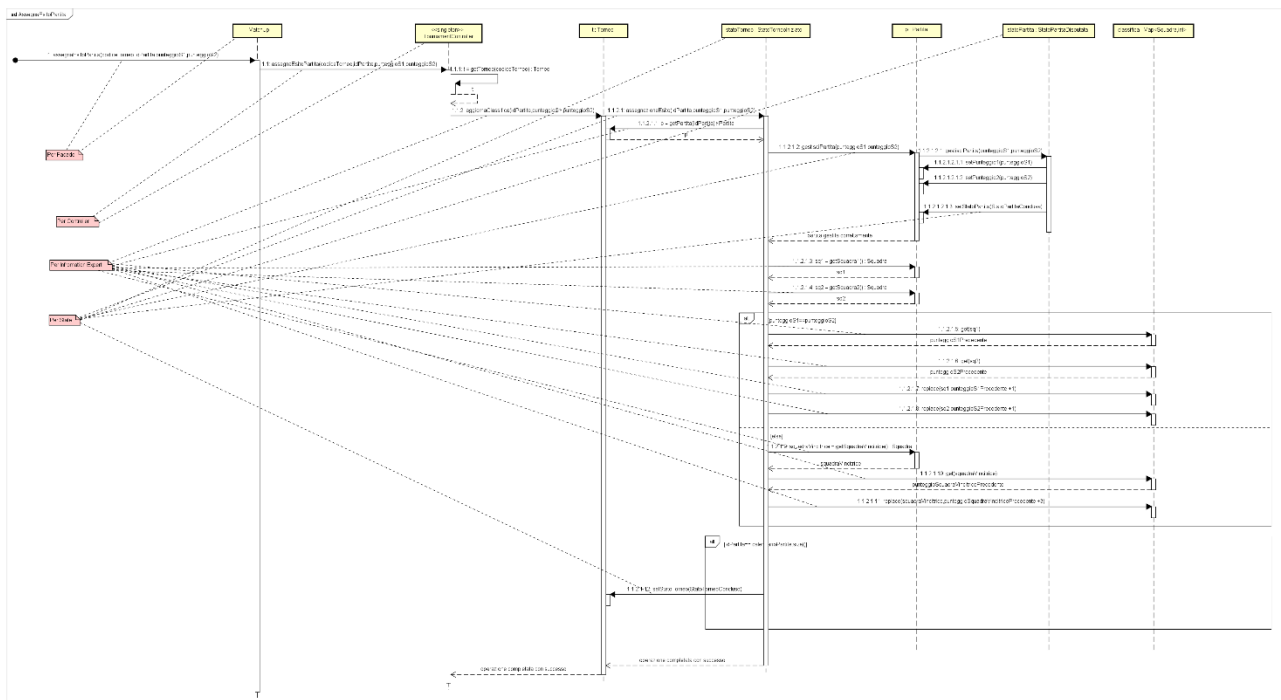


Figura 22: Diagramma di sequenza di assegna esito partita

3.2.11 ANNULLA PRENOTAZIONE

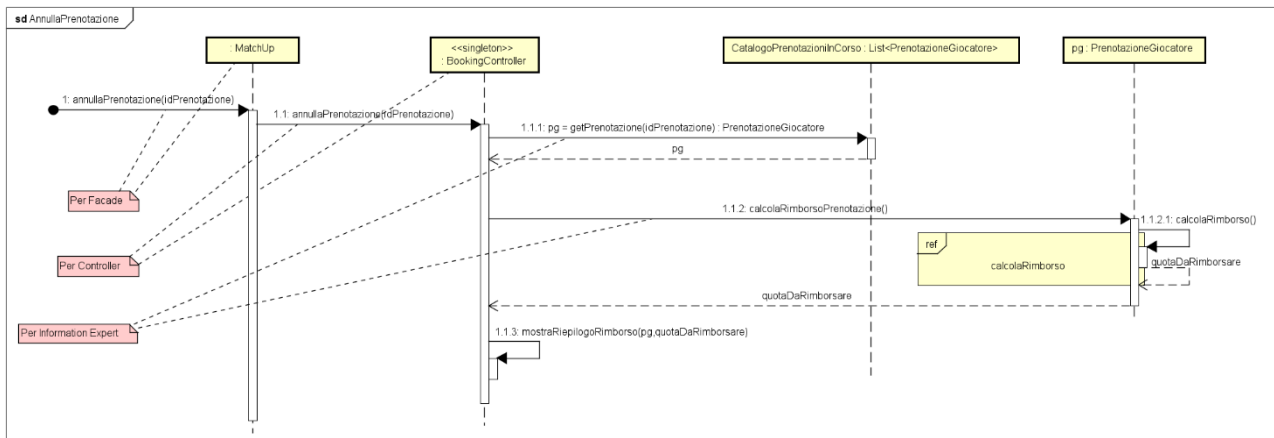


Figura 23: Diagramma di sequenza di annulla prenotazione

3.2.12 CALCOLA RIMBORSO

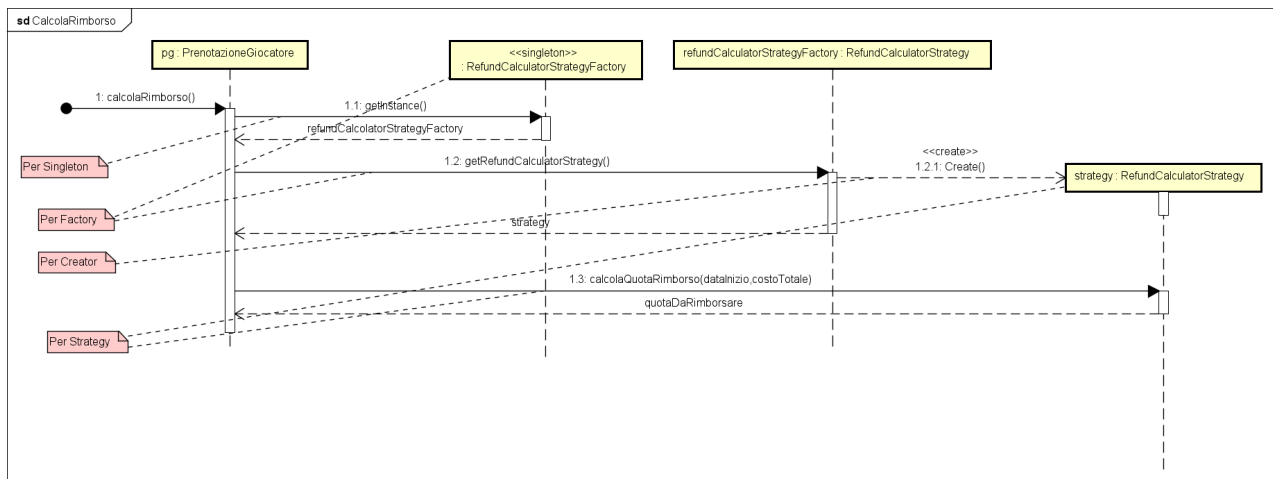


Figura 24: Diagramma di sequenza di calcola rimborsio

3.2.13 EFFETTUA RIMBORSO

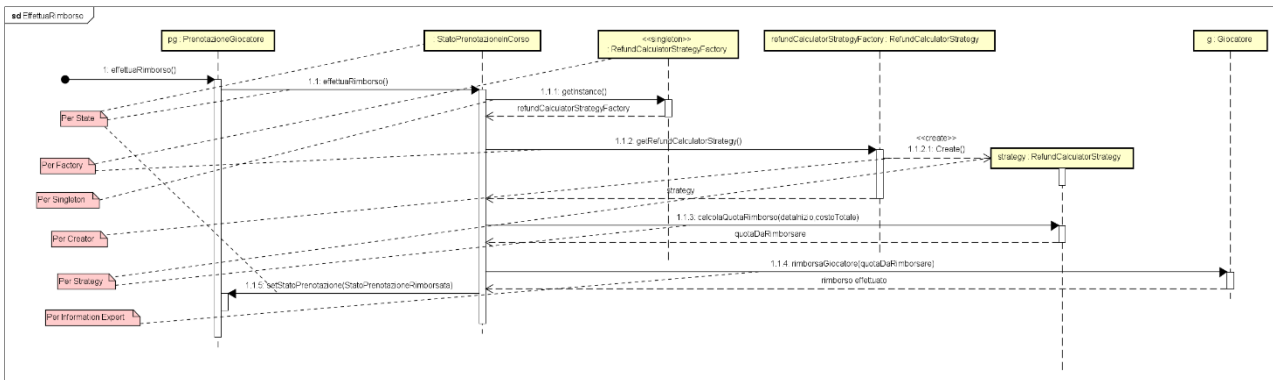


Figura 25: Diagramma di sequenza di calcola rimborsio

3.3 DIAGRAMMA DELLE CLASSI DI PROGETTO

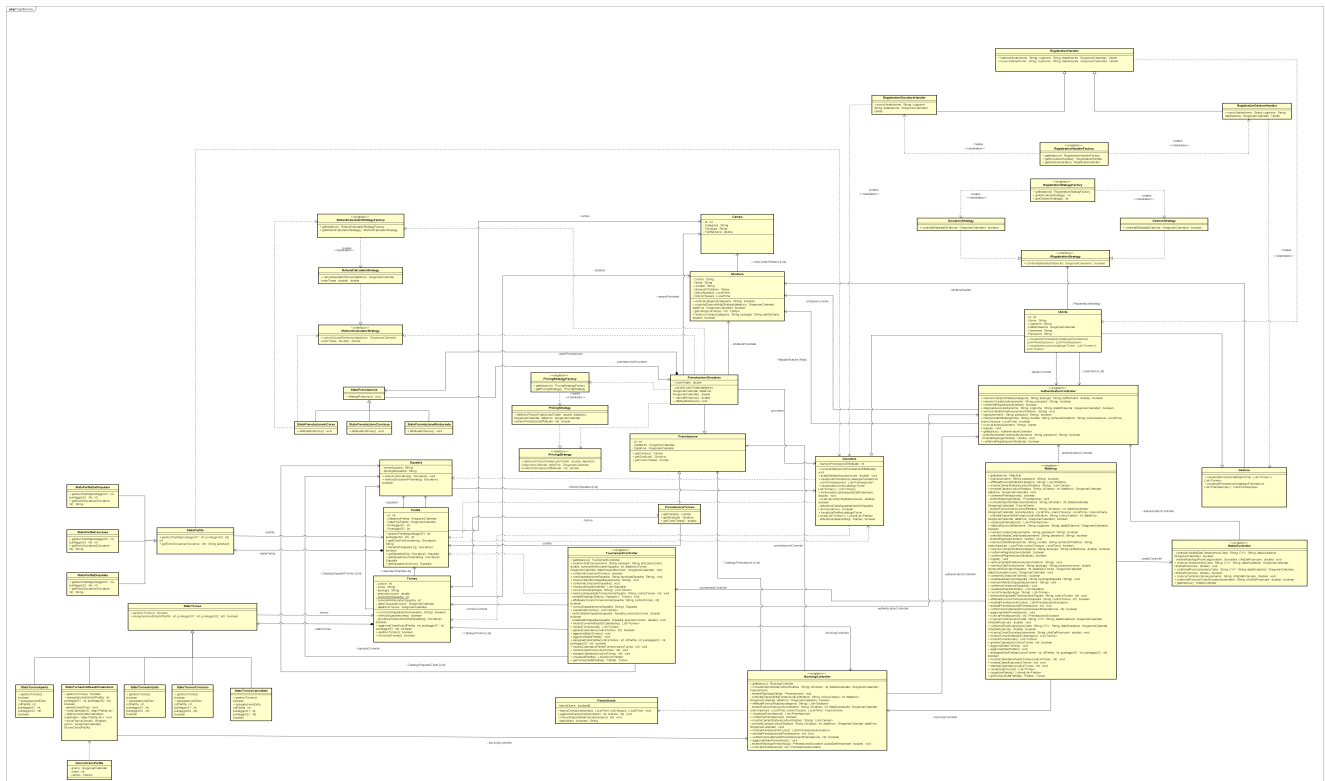


Figura 26: Diagramma delle classi di progetto

4 PATTERN UTILIZZATI NELLO SVILUPPO DEL PROGETTO

Durante l'implementazione dell'applicazione abbiamo utilizzato delle pratiche di progettazione per poter migliorare la qualità del codice e rendere il programma flessibile ad eventuali nuove aggiunte future. Sono stati utilizzati pertanto i seguenti pattern GRASP e GoF:

Controller: questo pattern è stato applicato in tutti quei contesti dove si necessitava di un oggetto coordinatore di un gruppo specifico di azioni. Si possono individuare pertanto vari oggetti che assumono il ruolo di controller, in particolare si possono individuare i controller di caso d'uso che si occupano di gestire una specifica funzionalità dell'applicativo. Tali controller sono:

- AuthenticationController: che si occupa della gestione dell'utente (registrazione e login)
- BookingController: che si occupa della gestione delle prenotazioni dell'utente
- TournamentController: che si occupa della gestione dei tornei
- WalletController: che si occupa delle operazioni di ricarica e accredito per gli utenti sul loro conto.

Facade: applicato sulla classe MatchUp: si tratta di un particolare controller che si occupa di ricevere

in ingresso i vari eventi dagli utenti e delegare la gestione di quest'ultimi a controller più specifici;

Strategy: questo pattern è stato applicato parecchie volte all'interno del progetto. Esso permette di modellare certe funzionalità che potrebbero in futuro essere soggette a modifiche. Si tratta di metodologie che implementano regole di dominio che a seconda della modificabilità di quest'ultime potrebbero indurre modifiche al codice. La strategy è stata applicata nelle seguenti funzionalità:

- Politiche di calcolo del prezzo delle prenotazioni;
- Controllo dell'età per la registrazione di gestori e giocatori;
- Politiche di rimborso delle prenotazioni;

Factory: questo pattern è stato applicato in coppia al pattern Strategy in quanto si è deciso di delegare il compito di istanziare gli oggetti di tipo strategy ad un oggetto intermediario;

Factory method: utilizzato per definire l'interfaccia di un metodo, che però verrà poi implementato dalle specifiche sottoclassi. Nel progetto, questo pattern è stato applicato al metodo nuovoUtente() che viene implementato dalle classi concrete (RegistrationGiocatoreHandler e RegistrationGestoreHandler) per istanziare l'oggetto specifico (Giocatore o Gestore).

Pure fabrication: applicato tramite la classe FasciaOraria per delegare la responsabilità di mostrare le fasce orarie disponibili ad un oggetto esterno, in modo da non far aumentare l'accoppiamento tra le classi Prenotazione, Struttura e Campo.

Singleton: applicato ai vari controller e alle factory di cui parlato sopra. Grazie a questo pattern abbiamo garantito una singola istanza di tali oggetti in memoria;

Polymorphism: applicato in diversi punti del sistema laddove il comportamento di un metodo varia in base al tipo concreto dell'oggetto, evitando il ricorso a controlli espliciti di tipo (instanceof) nel codice client. In particolare:

- sui metodi getStruttura(), getCampo() e getCostoTotale() della classe Prenotazione, che possiedono implementazioni diverse a seconda che si tratti di una prenotazione effettuata da un giocatore o di una prenotazione generata automaticamente per una partita di torneo;
- sui metodi visualizzaPrenotazioni() e visualizzaIscrizioni() della classe Utente, resi astratti e implementati diversamente dalle sottoclassi Giocatore e Gestore, ciascuna delle quali determina autonomamente la propria logica di filtro per individuare le prenotazioni e le iscrizioni di propria competenza.

State: applicato sulla classe Prenotazione del giocatore, in quanto essa può assumere diversi stati

durante la sua esistenza. Infatti, abbiamo valutato tre stati diversi:

- StatoPrenotazioneInCorso: viene impostato questo stato quando l'oggetto viene istanziato e permane finché la data di fine prenotazione non viene raggiunta;
- StatoPrenotazioneConclusa: viene impostato quando la data di fine prenotazione risulta trascorsa;
- StatoPrenotazioneRimborsata: viene impostato quando l'utente ha confermato l'annullamento della propria prenotazione. Tale pattern è stato applicato anche alla classe Torneo in quanto quest'ultimo evolve attraverso una serie di stati. A seconda, infatti, dello stato del torneo, cambiano le implementazioni dei metodi assegnazioneEsito() e gestisciTorneo().

Tali stati sono:

- StatoTorneoAperto: le iscrizioni al torneo sono aperte. Non è possibile gestirlo o assegnare esiti alle partite;
- StatoTorneoIniziato: non è più possibile iscriversi a questo torneo. È stato già generato il calendario ed è possibile assegnare l'esito alle partite, se la loro data risulta passata;
- StatoTorneoConcluso: il torneo è concluso e pertanto non è più modificabile;
- StatoTorneoCancellato: non sono state raggiunte le iscrizioni minime e quindi non è possibile gestire il torneo;
- StatoTorneoInAttesaDiCalendario: il torneo ha raggiunto le iscrizioni minime ed esso è in attesa della generazione del calendario.

L'implementazione di questo pattern, infine, lo si ha pure nella classe Partita, in quanto anche quest'ultima risulta essere un oggetto dinamico che evolve attraverso i seguenti stati:

- StatoPartitaDaDisputare: è lo stato iniziale dell'oggetto. Si mantiene questo stato finché la data di fine dell'incontro non risulta superata;
- StatoPartitaDisputata: si arriva in questo stato dopo che la data di fine incontro risulta trascorsa e la partita si trova in attesa di assegnazione del risultato da parte del gestore;
- StatoPartitaConclusa: si arriva in questo stato dopo che il gestore registra l'esito della partita.

5 FASE DI TESTING

5.1 INTRODUZIONE AL TESTING

Il testing rappresenta una delle attività fondamentali nel processo di sviluppo software, poiché consente di verificare il maggior numero possibile di casistiche e di ottenere una visione completa del comportamento dell'applicazione nelle diverse condizioni operative.

Il progetto è stato sottoposto a test attraverso JUnit, un framework Java ampiamente diffuso per l'automazione dei test. Con questo strumento sono stati realizzati i test funzionali, ossia test mirati ai singoli metodi, volti a verificare che, a fronte di determinati input, il sistema restituisca esattamente i risultati attesi. Questa tipologia di test si implementa attraverso apposite classi di test, all'interno delle quali sono definiti metodi contenenti chiamate alle funzioni di assert messe a disposizione dal framework: tali funzioni confrontano il risultato atteso con quello effettivamente prodotto dal codice, segnalando eventuali discrepanze.

La selezione dei metodi da sottoporre a test si è concentrata su quelli con maggiore rilevanza all'interno del sistema. In particolare, la strategia adottata non si è limitata a testare esclusivamente i metodi che implementano direttamente un caso d'uso o una macrofunzionalità, ma ha previsto anche la verifica dei metodi da questi richiamati internamente. Questo approccio ha permesso di ottenere un riscontro progressivo e granulare sul corretto funzionamento delle singole componenti che concorrono a realizzare ciascuna funzionalità.

Accanto ai test funzionali, è stata prevista una seconda categoria di test, i test di sistema, che valutano invece il comportamento dell'applicazione nel suo complesso.

5.2 SCENARIO REGISTRAZIONE GIOCATORE

In questo scenario un giocatore desidera registrarsi al servizio MatchUp. Lo scenario prevede i seguenti passaggi:

- Inserimento dei dati anagrafici del giocatore: nome, cognome e data di nascita.
- Verifica dell'età del giocatore.
- Conferma da parte del giocatore dei dati inseriti.
- Scelta di uno username.
- Scelta di una password.

Il metodo `istanziaUtente()`, implementato nella classe `MatchUp`, si occupa di creare l'istanza tramite il `RegistrationHandler` appropriato e di verificare, tramite la relativa `IRegistrationStrategy`, che l'età sia superiore al minimo di legge (14 anni per i giocatori, secondo la regola di dominio R6).

I passaggi 4 e 5 richiedono al giocatore di scegliere uno username e una password validi, secondo le seguenti condizioni:

- lo username deve essere univoco nel sistema;

- la password deve essere lunga almeno 8 caratteri e seguire regole di formattazione.

In definitiva, tramite test di tipo unitario vengono testati i seguenti metodi:

- **istanziaUtente()** ,implementato nella classe MatchUpTest, verificando i seguenti scenari:

- a) registrazione di un giocatore con età inferiore a 14 anni (scenario di fallimento);
- b) registrazione di un gestore con età inferiore a 18 anni (scenario di fallimento);
- c) registrazione di un giocatore con età superiore a 14 anni (scenario di successo);
- d) registrazione di un gestore con età superiore a 18 anni (scenario di successo).

- **confermaRegistrazione()** / **confermaRegistrazioneGestore()**, verificando i seguenti scenari:

- a) non è presente alcuna registrazione in corso (scenario di fallimento);
- b) è presente una registrazione in corso (scenario di successo), permettendo il salvataggio dell'utente nel sistema.

- **inserisciCredenziali()**, verificando i seguenti scenari:

- a) username già presente nel sistema (scenario di fallimento);
- b) username non presente nel sistema (scenario di successo);
- c) password con lunghezza inferiore a 8 caratteri o con caratteri non validi (scenario di fallimento);
- d) password con lunghezza uguale o superiore a 8 caratteri e che rispetta le regole di formattazione (scenario di successo).

5.3 SCENARIO REGISTRAZIONE STRUTTURA

Lo scenario complementare a quello del giocatore è la registrazione del gestore e della sua struttura. I passaggi Di inserimento e convalida dei dati anagrafici e non, sono identici a quelli descritti nello scenario 5.2, con l'unica differenza che l'entità istanziata è di tipo Gestore anziché Giocatore, e non verranno pertanto ripetuti. Lo scenario prosegue poi con:

- Inserimento dei dati della struttura: nome, località, numero di telefono, orario di apertura e orario di chiusura.
- Inserimento dei dati di uno o più campi della struttura: categoria, tipologia e tariffa oraria. Il passo viene ripetuto finché il gestore lo desidera.
- Conferma dei dati inseriti.

Il metodo **inserisciDatiStruttura()** si occupa di istanziare la struttura e di associarla come struttura corrente; il metodo **confermaRegistrazioneStruttura()** la associa definitivamente a MatchUp e al gestore che l'ha registrata.

Vengono quindi testati:

- **inserisciDatiStruttura()** e **inserisciCampoStruttura()**, verificando la corretta creazione e associazione delle istanze di Struttura e Campo.
- **confermaRegistrazioneStruttura()**, verificando i seguenti scenari:
 - a) non è presente alcuna registrazione di struttura in corso (scenario di fallimento);
 - b) è presente una registrazione di struttura in corso, associata a un gestore correttamente registrato (scenario di successo).

5.4 SCENARIO PRENOTA CAMPO

Lo scenario di prenotazione del campo è stato aggiornato per includere l'autenticazione del giocatore, un controllo aggiuntivo sull'orario di apertura della struttura, la gestione dettagliata delle fasce orarie e il calcolo del costo basato sullo storico delle prenotazioni. Lo scenario di test copre l'intero ciclo di vita della prenotazione, articolato nei seguenti passaggi.

Passaggio 1: Operazione di Login: Prima ancora di tentare una prenotazione, viene testata l'operazione di login() come funzionalità a sé stante all'interno della classe MatchUpTest, essenziale per l'autenticazione di gestori e giocatori.

- **Credenziali errate:** l'inserimento di uno username inesistente o di una password non corrispondente restituisce false e l'attributo utenteCorrente rimane null.
- **Credenziali corrette:** l'inserimento di una coppia username/password valida restituisce true l'istanza corrispondente viene correttamente assegnata a utenteCorrente.

Passaggio 2: Ricerca, Visualizzazione Strutture e Campi: Avviene tramite i metodi effettuaRicercaStruttura(categoria) e mostraCampiStruttura(codiceStruttura) testati nella classe MatchUpTest.

- **Test su effettuaRicercaStruttura:** verifica il filtraggio. Restituisce una lista popolata con le strutture idonee se esistono corrispondenze; restituisce una lista vuota nel caso in cui nessuna struttura contenga campi per la categoria specificata.
- **Test su mostraCampiStruttura:** verifica che, fornito il codice di una struttura esistente, il sistema restituisca correttamente la lista delle istanze di Campo ospitate da essa. Fornendo un codice struttura inesistente, il metodo restituisce una lista vuota o solleva un'eccezione.
- **Test su verificaCategoria (effettuato in StrutturaTest):** Verifica se la struttura possiede almeno un campo della categoria cercata. Esiti: true se presente, false altrimenti.

Passaggio 3: Richiesta Disponibilità e Gestione Fasce Orarie: Questo passaggio testa la visualizzazione degli orari liberi tramite il metodo pubblico `richiediDisponibilità()`, che internamente richiama il metodo privato `mostraFasceOrarie()` per costruire l'oggetto `FasciaOraria` risultante. Quest'ultimo filtra dal catalogo globale le sole prenotazioni relative alla struttura, al campo e al giorno richiesto, aggiornando di conseguenza le fasce orarie occupate tramite il metodo `aggiornaFasciaOraria()`. La verifica avviene per confronto: viene costruito manualmente un oggetto `FasciaOraria` di riferimento, applicando le medesime modifiche corrispondente alle prenotazioni note per quel campo e quella data, e il suo output viene confrontato con quello restituito dal sistema. Vengono verificati i seguenti scenari:

a) Prenotazioni multiple nello stesso giorno: a fronte di più prenotazioni esistenti per il medesimo campo e la medesima data, le relative fasce orarie risultano correttamente marcate come occupate, mentre le restanti rimangono disponibili. Un ulteriore controllo verifica che, applicando all'oggetto di riferimento una modifica corrispondente a una prenotazione non realmente esistente nel sistema, il confronto con l'output del sistema risulti correttamente negativo.

Passaggio 4: Generazione e Validazione della Prenotazione (`prenotaCampo`): Il metodo `prenotaCampo()` rappresenta la transizione di stato principale nel processo di prenotazione e gestisce la validazione sequenziale dei requisiti di dominio:

- **a) Verifica Autenticazione:** se `utenteCorrente` è null oppure non è un'istanza di `Giocatore` (ad esempio in caso di login effettuato da un `Gestore`), il metodo interrompe l'esecuzione e restituisce false.
- **b) Codice Struttura inesistente:** se il codice della struttura non trova riscontro all'interno della collezione `mappaStrutture`, l'esecuzione si interrompe restituendo false.
- **c) Codice Campo inesistente:** se la struttura esiste ma il metodo `getCampo(idCampo)` restituisce null, l'esecuzione si interrompe restituendo false.
- **d) Struttura non aperta nella fascia oraria richiesta:** se l'orario di inizio o di fine della prenotazione non rientra nell'intervallo compreso tra l'orario di apertura e di chiusura della struttura, la richiesta viene respinta e il metodo restituisce false.
- **e) Fascia oraria del campo già occupata:** se il campo risulta già prenotato nella medesima data e fascia oraria (rilevando una sovrapposizione con le prenotazioni già confermate), l'operazione fallisce e il metodo restituisce false.
- **f) Esito positivo:** se tutti i controlli antecedenti hanno esito positivo, `MatchUp` crea

l'oggetto della prenotazione, associandolo temporaneamente a prenotazioneCorrente in attesa di conferma, e restituisce true.

Passaggio 5: Conferma e Persistenza (confermaPrenotazione) Il metodo confermaPrenotazione() chiude il flusso.

- a) **Esecuzione fuori sequenza:** se prenotazioneCorrente è null, restituisce false.
- b) **Conferma con dati coerenti:** il metodo restituisce true.

5.5 SCENARIO DI CREAZIONE TORNEO

La funzionalità consente a un gestore autenticato di configurare e pubblicare un nuovo torneo associato alla propria struttura sportiva. L'iter di esecuzione è articolato nelle seguenti fasi:

1. Immissione dei parametri di configurazione del torneo da parte dell'utente;
2. Visualizzazione della schermata di riepilogo delle informazioni inserite;
3. Validazione definitiva e salvataggio del torneo.

Durante il primo step, prima ancora dell'invocazione della logica di business, vengono eseguiti dei controlli sintattici primari all'interno della MainClass: si verifica che valori numerici come il costo di iscrizione e la quota minima di componenti per squadra siano rigorosamente maggiori di zero. Qualora la validazione iniziale vada a buon fine, il sistema invoca il metodo inserisciDatiTorneo() esposto dalla classe ControllerTornei.

La suite di test unitari per questo metodo analizza i seguenti scenari:

- **a) Scenario di successo:** Un gestore *ge* crea un torneo relativo a una disciplina sportiva effettivamente erogata dalla struttura *s* di sua competenza;
- **b) Scenario di fallimento:** Un gestore *ge* prova a registrare un torneo per uno sport non previsto tra le attività della struttura *s*;
- **c) Scenari di fallimento** Tenta la creazione un utente con ruolo "Giocatore" o un utente non autenticato. (è uno scenario non realizzabile perché solo il gestore loggato può fare questa operazione)

Nota: Sebbene lo scenario *c* sia reso inaccessibile a livello di flusso dell'interfaccia la risposta del metodo a chiamate non autorizzate è stata comunque collaudata con esito positivo per garantire la sicurezza a livello di controller.

Infine, la fase 3 trova attuazione nel metodo confermaCreazioneTorneo(), il quale restituisce un esito positivo esclusivamente se nel contesto di esecuzione risulta attiva una sessione di creazione torneo.

5.6 SCENARIO DI CREAZIONE DELLA SQUADRA

Spostando l'attenzione sul versante dell'utente finale (giocatore), è stata implementata la funzionalità che permette di comporre una squadra e iscriverla ai tornei disponibili. Il flusso operativo di creazione della squadra prevede:

1. Definizione dell'identità della squadra (inserimento del nome e della disciplina sportiva);
2. Popolamento della rosa con l'inserimento dei giocatori;
3. Salvataggio finale e registrazione della squadra.

A fronte del primo passaggio, viene eseguito il metodo `creaSquadra()` del `ControllerTornei`. Il test unitario valuta in questo punto tre differenti condizioni:

- **a) Scenario di fallimento:** Tentativo di utilizzo di un nome squadra già esistente nel sistema;
- **b) Scenario di fallimento (non raggiungibile):** Esecuzione del metodo in assenza di sessione attiva o da parte di un utente con profilo Gestore;
- **c) Scenario di successo:** Inserimento di un nome squadra univoco e non ancora presente a sistema.

In caso di esito positivo al passo 1, l'utente proprietario della sessione (il creatore) viene incluso automaticamente tra i componenti della squadra. Successivamente, la fase di inserimento degli altri membri avviene richiedendo iterativamente l'username di ciascun partecipante. L'operazione è gestita dal metodo `inserisciMembroSquadra()`, sottoposto ai seguenti test:

- **a) Scenario di fallimento:** l'username digitato non corrisponde a un utente registrato con ruolo Giocatore;
- **b) Scenario di fallimento:** L'username risulta non essere nel sistema;
- **c) Scenario valido:** l'username identifica un Giocatore registrato e valido.

Il superamento del caso *c* non completa direttamente l'inserimento: la logica viene delegata al metodo `inserisciGiocatore()` della classe `Squadra`, il quale effettua un ulteriore controllo di ridondanza per evitare che lo stesso giocatore venga aggiunto più volte alla stessa squadra.

L'operazione si conclude con la fase 3 (conferma e persistenza della squadra), i cui casi di errore ricalcano le medesime condizioni di fallimento individuate per i restanti metodi di conferma.

5.7 SCENARIO DI ISCRIZIONE AL TORNEO

Una volta completata la fase di creazione della squadra, ciascun giocatore appartenente alla rosa ha la possibilità di formalizzare la partecipazione della squadra a un torneo le cui iscrizioni risultino ancora aperte. L'elaborazione è affidata al metodo `effettualiscrizioneTorneo()` presente nel `ControllerTornei`.

Gli scenari posti sotto test per questa procedura verificano contestualmente che:

- **a)** La richiesta di iscrizione pervenga entro i termini e le date limite stabiliti dal torneo;
- **b)** L'utente che sottoscrive l'iscrizione sia effettivamente presente all'interno della squadra selezionata;
- **c)** Il giocatore che esegue l'operazione disponga di un saldo personale sufficiente a coprire la quota d'iscrizione;
- **d)** La disciplina sportiva della squadra coincida perfettamente con lo sport previsto dal torneo.
- **e):** Nessun componente della squadra risulti già iscritto allo stesso torneo all'interno della rosa di un'altra squadra già iscritta.

Solo qualora tutti i requisiti sopra descritti diano esito positivo, il sistema procede all'addebito dell'importo previsto sul saldo del giocatore e conferma l'iscrizione della squadra al torneo.

5.8 SCENARIO DI ANNULLAMENTO PRENOTAZIONE

Lo scenario annullamento di prenotazione prevede la seguente sequenza di passaggi:

1. Ricerca della prenotazione da annullare;
2. Conferma annullamento prenotazione;
3. Emissione del rimborso della prenotazione (se previsto);
4. Liberazione della fascia oraria;

Dopo aver recuperato in ingresso dall'utente il codice della prenotazione da annullare, viene chiamato il metodo *annullaPrenotazione()* implementato nella classe *BookingController*. Se la prenotazione viene trovata, viene calcolato il rimborso in accordo alle regole di dominio e mostrato il riepilogo al giocatore.

Se l'annullamento viene confermato, tramite la chiamata al metodo *effettuarimborso()*, implementato nello *StatoPrenotazione*, viene emesso il riaccredito del rimborso al giocatore e liberata la fascia oraria. Affinché il rimborso avvenga correttamente, viene verificato che lo stato della prenotazione deve essere "in corso e non a conclusa oppure già rimborsata". Gli scenari di test individuati sono:

- a) Annullamento di una prenotazione esistente ed in corso. È lo scenario di successo.
- b) Annullamento di una prenotazione non esistente. È uno scenario di fallimento.
- c) Annullamento di una prenotazione esistente ma con stato non "in corso". È uno scenario di fallimento.

5.9 SCENARIO RICARICA CONTO

Lo scenario di ricarica conto permette al giocatore attualmente autenticato al servizio di ricaricare in autonomia il proprio conto. I passaggi da eseguire sono i seguenti:

1. Richiesta da parte dell'utente dei seguenti dati: cifra da ricaricare, numero di carta, data scadenza carta, CVV.
2. Verifica correttezza dati inseriti e generazione del riepilogo ricarica;
3. Conferma della ricarica.

Dopo aver eseguito il passaggio 1, viene chiamato il metodo *ricaricaConto()* implementato nella classe *WalletController*: tale metodo, dopo aver verificato i dati da parte del metodo *controllaValiditàDatiCarta()* implementato nella medesima classe, emette il riepilogo della ricarica carta. Il passaggio 3 richiede la conferma al giocatore del pagamento ed in caso di esito positivo si ha un accredito dell'importo sul conto del giocatore. Gli scenari di test individuati sono:

- a) Il numero di carta inserito non è appartenente ai seguenti circuiti: Mastercard, Visa, American Express. È uno scenario di fallimento.
- b) Il CVV non ha il formato corretto. È uno scenario di fallimento.
- c) La data di scadenza della carta è stata superata. È uno scenario di fallimento.
- d) L'importo inserito è superiore a 300 € o inferiore a 10 €.
- e) I dati della carta sono corretti e la ricarica viene effettuata. È lo scenario di successo.

5.10 SCENARIO DI RICARICA CONTO GIOCATORE

Questo scenario di testing riguarda la ricarica da parte dei gestori della struttura, dei conti dei vari giocatori. Questo scenario è un'alternativa del precedente, in quanto nel caso un giocatore non possieda una carta di credito egli può comunque usare il servizio. Il gestore, a richiesta del giocatore, si fa comunicare l'username e la cifra che vuole ricaricare. A seguito della conferma la cifra viene accreditata. I passaggi da eseguire in questo scenario sono i seguenti:

1. Richiesta dell'username e della cifra da ricaricare;
2. Inserimento dei dati richiesti e generazione del riepilogo della ricarica;
3. Conferma della ricarica.

Il metodo che si occupa di effettuare la ricarica al conto del giocatore è *ricaricaContoGiocatore()* implementato nella classe *WalletController*. Gli scenari di test individuati sono:

- a) Username inserito inesistente. È uno scenario di fallimento;
- b) Importo inserito non valido. È uno scenario di fallimento;

c) Username esiste e importo valido. È lo scenario di successo.

5.11 MODIFICA SCENARIO DI TEST ISCRIZIONE TORNEO

Precedentemente era stato valutato lo scenario di test di iscrizione al torneo, che in questa sezione ha però subito una variazione per quanto riguarda il raggiungimento dello scenario di successo. Ecco le condizioni aggiornate:

- **a)** La richiesta di iscrizione pervenga entro i termini e le date limite stabiliti dal torneo;
- **b)** L'utente che sottoscrive l'iscrizione sia effettivamente presente all'interno della squadra selezionata;
- **c)** Tutti i giocatori che fanno parte della squadra dispongano di un saldo personale sufficiente a coprire la quota d'iscrizione, dove la quota individuale è calcolata come **Prezzoliscrizione / numero giocatori in squadra**;
- **d)** La disciplina sportiva della squadra coincida perfettamente con lo sport previsto dal torneo.
- **e):** Nessun componente della squadra risulti già iscritto allo stesso torneo all'interno della rosa di un'altra squadra già iscritta.

La modifica della condizione **c** è stata fatta per due motivi:

1. Nel caso di cancellazione del torneo per numero insufficiente di squadre, verrà effettuato un rimborso a tutti gli utenti della squadra e questo elimina la necessità di salvare l'identificativo del giocatore che ha effettuato l'iscrizione;
2. È più pratico, anzi più corretto che i vari giocatori che compongono la squadra contribuiscano al pagamento della quota di iscrizione.

Inoltre, sempre in questa iterazione i test prevedono il meccanismo di ricarica conto in caso di saldo insufficiente, che giustamente è stato introdotto precedentemente.

5.12 SCENARIO DI GENERAZIONE DEL CALENDARIO DEL TORNEO

La generazione del calendario di un torneo rappresenta un meccanismo complesso dell'applicazione MatchUp in quanto rappresenta un sistema automatizzato per l'organizzazione di un torneo. Il flusso di esecuzione del metodo `generaCalendario()`, implementato nella classe `TournamentController`, è il seguente:

1. Inserimento del codice del torneo da gestire;
2. Verifica che sia trascorsa la data di scadenza delle iscrizioni e che le squadre iscritte siano superiori al numero minimo (pari a 8 come impostato nelle regole di dominio);

- a. Se le squadre iscritte è sono inferiori a 6 viene rimborsata la quota d'iscrizione a tutte le squadre;
- b. Altrimenti viene generato il calendario delle partite, inserendo nel sistema le prenotazioni per i campi.

L'algoritmo utilizzato per generare il calendario è l'algoritmo di Berger, un metodo matematico utilizzato per generare automaticamente il calendario delle giornate nei tornei "all'italiana" (girone *round-robin*), garantendo che ogni partecipante affronti tutti gli altri una sola volta. Gli scenari di fallimenti individuati per questo metodo consistono nei tentativi di generazione del calendario per tornei che sono conclusi, iniziati oppure cancellati; pertanto, nella classe di test rispettiva è possibile visionare come la chiamata di questo metodo su tornei con tali stati ritorna esito *false*. Per accertare invece il funzionamento nello scenario di successo, oltre a verificare che il valore di ritorno sia *true*, si verifica che il numero di prenotazioni per quel determinato torneo coincida con il numero delle partite.

5.13 SCENARIO DI ASSEGNAZIONE ESITO ALLA PARTITA

Dopo aver generato il calendario delle partite, deve essere possibile poter assegnare i punteggi alle squadre che giocano le partite del torneo e dunque assegnare i punti in modo da automatizzare la generazione della classifica.

Questo scenario viene gestito dal metodo `assegnazioneEsitoPartita()` implementato nella classe `TournamentController`. Il flusso di esecuzione di questo scenario è il seguente:

1. Inserimento del codice del torneo e dell'id della partita da gestire;
2. Inserimento dei punteggi delle due squadre
3. Verifica che lo stato del torneo sia iniziato e che lo stato della partita sia disputata e in caso affermativo assegnare i punteggi alle squadre della partita svolta e alla squadra vincitrice assegnare i punti vittoria o il punto pareggio ad entrambe le squadre per poter aggiornare la classifica;
4. Se inoltre la partita disputata è anche l'ultima, allora la squadra con più punti nella classifica vincerà il torneo, il quale sarà concluso

Gli scenari di fallimento individuati sono i seguenti:

- a) Assegnazione esito di una partita di un torneo inesistente;
- b) Assegnazione esito di una partita di un torneo con iscrizioni ancora aperte;
- c) Assegnazione esito di una partita di un torneo iniziato ma con partita non ancora disputata;
- d) Assegnazione esito di una partita di un torneo iniziato ma con partita conclusa.

Nel caso di scenario di successo, sono stati effettuati dei test sulla corretta assegnazione dei punti in classifica in base ai punteggi ottenuti nella partita disputata. E' stato verificato quindi:

- a) L'assegnazione dei 3 punti in classifica per la squadra vincitrice (regola di dominio);
- b) L'assegnazione di 1 punto in classifica per entrambe le squadre, in caso di pareggio (regola di dominio).